PDDL-Based Mission Planning for a Drone Swarm Simulator

MTP Project Report â€” Planning Component

Submitted by: [Student name]

Roll number: [Roll number]

Department: [Department]

Institution: [Institution]

Supervisor: [Supervisor name]

Date: 23 October 2026

This report documents the PDDL planning component in the supplied simulator workspace. Personal details must be completed before submission. The structure is adapted from Reference Doc.pdf; its sample PhD title, institution, certificate, publications and personal statements have not been treated as project facts.

Abstract

This project provides a planning layer for a drone swarm simulator. Planning Domain Definition Language (PDDL) describes the conditions under which a drone may move, join a formation, search an area, perform a show, or complete a mission. A domain file defines the available actions. A problem file defines the drones, locations, initial conditions and mission goals. The simulator prepares a problem from map inputs and sends it to the Expressive Numeric Heuristic Search Planner (ENHSP). The planner returns an ordered sequence of actions. Python services convert relevant actions into geographic routes and, where required, separate routes and timing information for each drone.

The current workspace contains ten mission types: point-to-point travel, forest search, V formation, two-dimensional grid formation, three-dimensional grid formation, parabola formation, figure-eight formation, star formation, planetary orbits and road inspection. This report explains their models, action sequences, completion conditions, communication relationships and integration with execution. It distinguishes symbolic planning conditions from actual network communication and flight control. The report describes the implementation and its limits; it does not claim measured performance or flight validation that was not performed during this documentation task.

Keywords: PDDL, drone swarm, automated planning, ENHSP, formation control, mission planning, simulation.

Contents

1. Introduction and scope

2. Planning concepts and system architecture

3. Mission models

4. Communication and coordination

5. Safety, failures and model limitations

6. Execution workflow and evaluation method

7. Conclusions and future work

References

Appendix A. Complete PDDL source listings

List of tables and figures

Table 3.1: Mission inventory. Table 4.1: Neighbour relationships. Table 6.1: Proposed validation cases. Figure 2.1: Planning and execution flow. Figure 4.1: Communication paths.

Glossary, abbreviations and notation

PDDL: Planning Domain Definition Language. ENHSP: Expressive Numeric Heuristic Search Planner. GCS: Ground Control Station. GUI: Graphical User Interface. UDP: User Datagram Protocol. MAVLink: the flight communication protocol used by the mission execution components. SITL: Software in the Loop. SYSID: drone system identifier.

Predicate: a true or false statement about the planning state. Numeric fluent: a numeric value that may change after an action. Precondition: a condition required before an action. Effect: a state change caused by an action. Ground action: an action whose parameters have been replaced by actual object names. Corridor: the leader's geographic route. Slot: a drone's assigned place relative to a formation or path. Neighbour: a drone connected by a defined logical relationship.

N denotes drone count. B denotes battery level and E denotes action energy cost. R and C denote grid rows and columns. L denotes the number of vertical layers. Distances and offsets in the geometry code are in metres. Battery values are model units unless explicitly calibrated; a value of 100 does not establish a physical capacity in watt-hours.

1. Introduction and scope

The planning problem is to decide which actions can achieve a drone mission while respecting the conditions represented in the model. A route alone is insufficient: a drone may also need adequate battery, healthy GPS, a working communication link and the correct formation state. PDDL makes these requirements explicit and allows a planner to search for an action sequence that satisfies them.

The planning contribution consists of the domain and problem models, their mission rules, numeric resource conditions, formation readiness conditions and completion goals. The surrounding simulator prepares geographic inputs, requests a plan, extracts routes and executes flight commands. These supporting components are described where they explain how a PDDL plan becomes a simulated mission. Their presence does not imply that every component was authored as part of the student's individual contribution.

The report covers the current files in plans/ and the integration code in services/ and engine/. It uses the reference PDF as a structural example, rather than as a technical source for drone behaviour. The existing four mission PDFs in docs/ are earlier mission-specific documents; current source files are the authority for the ten-mission inventory. Archived run summaries are historical evidence and are not presented as new experimental results.

2. Planning concepts and system architecture

2.1 Domain and problem files

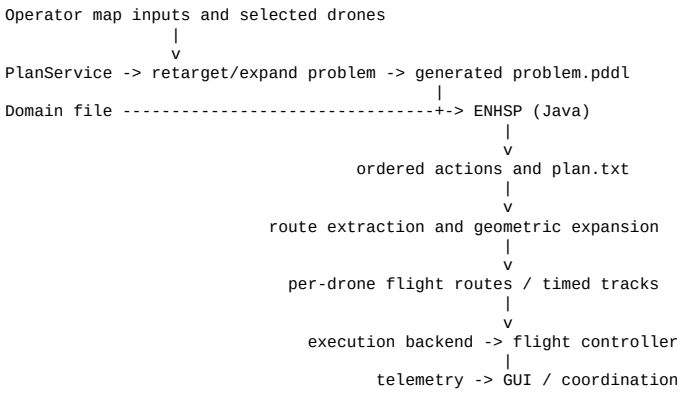
A domain file defines object types, predicates, numeric functions and actions. A problem file refers to that domain and provides concrete objects, initial facts, numeric values and goals. The planner searches from the initial state to a state satisfying every goal condition. A goal is a logical condition, not an instruction sent directly to an aircraft.

For example, (at drone1 source) states that drone1 is at the source. (communication-ok drone1) represents a working communication condition. (= (battery drone1) 85) sets an initial battery value. A movement action can require B > E, remove the old location, add the new location and reduce battery to B - E. A strict greater-than condition rejects a move when battery exactly equals the required energy.

The models use typed objects and numeric fluents. Formation models also use equality, negative and disjunctive conditions, universal conditions and conditional effects. A universal condition can require every formation member to be ready. A conditional effect can update all objects having the required role. These are discrete state transitions. The files do not use durative actions; a numeric launch counter does not by itself measure wall-clock seconds.

2.2 From map selection to flight

Figure 2.1. Planning and execution flow



services/plan_service.py discovers folders containing a domain and a problem file. It identifies the mission kind using the domain name inside the file. Search uses SearchProblem.pddl; the other current templates use problem.pddl. Problem preparation can change coordinates, add drones, rebuild neighbour facts and recompute route energy. The generated problem is stored in plans/<mission>/runs/<timestamp>/problem.pddl. Planner output is stored beside it as plan.txt.

engine/pddl_planner.py locates Java and the ENHSP JAR, starts the planner as a subprocess and parses action lines into PlanStep objects. A step contains an index, action name and arguments. The default timeout is 30 seconds, with some mission-specific calls allowing longer. ENHSP_JAR can override the default tools/enhsp/enhsp-dist/enhsp.jar location. The worker runs outside the GUI thread and reports completion or failure through Qt signals.

Route extraction depends on the domain. Travel uses travel actions. Search uses move-search and return-to-base. Formation corridors use formation-cruise. The geometric code derives offset member routes, curved paths or timed show tracks. PDDL therefore plans mission logic and selected movement structure; Python supplies detailed geometric information that is not fully represented by symbolic waypoint names.

2.3 Meaning of formation locations

Several models assign the leader and members to the same symbolic location after a coordinated move. This means they share a formation reference waypoint. It does not mean that aircraft should occupy identical physical coordinates. Along-track, cross-track, arc-length and altitude offsets separate their actual routes. The geometric conversion is essential to understanding these models.

3. Mission models

Each mission section explains its purpose, its normal action flow, the result required for completion and its communication assumptions. The action descriptions are extracted from the current domain comments. Appendix A contains every domain and problem file verbatim, including numeric values, preconditions and effects.

Table 3.1. Mission inventory

Folder	Mission	Main operator inputs
travell	Point-to-point travel	Source, destination, optional restricted area
Search	Forest or area search	Base, search polygon, selected drones
vformation	V formation	Source, destination, selected drones
gridformation	Horizontal grid	Source, destination, selected drones
3dgridformation	Layered grid or cube	Source, destination, selected drones
parabolaformation	Three-drone parabola	Source and destination
figure8formation	Figure-eight train	Base, left end, crossing, right end, laps
starformation	Circle and star show	Show centre, selected drones, laps
planets	Planetary orbit show	Sun centre, selected drones, laps
Inspect	Road inspection	Source and destination on a road

3.1 Point-to-point travel

Source folder: plans/travell/. Domain name: swarm-drone-mission.

The swarm travels from a chosen source to a destination. The reference drone is expanded for the selected fleet. Routes may include intermediate waypoints to avoid a marked restricted area. The model uses connected and safe-route facts, distance checks, battery checks and mission-completed conditions.

The normal sequence is check-distance, check-battery, plan-safe-route, travel and reach-destination. Movement consumes the energy for that edge and clears route-planned, so another planning action is required for the next movement. The template starts drone1 at battery 85, with minimum-return-battery 30 and health 95. These are sample instance values, not fixed rules for every mission.

The goal requires drone1 at the destination with mission-completed. The template does not require a final landed state. Failure actions represent warnings, controller notification, return, hover and a sacrifice state. The sacrifice-drone action is an abstract model outcome, not evidence of a physical destruction procedure. No fixed drone-to-drone neighbour mesh is defined for this mission.

Action catalogue (source order; this is not a newly solved plan):

- check-distance: Check the distance before planning the route

- check-battery: Check whether the drone has enough battery

- notify-insufficient-battery: Record that the battery is not enough

- plan-safe-route: Create a route only when all safety checks pass

- travel: Move the drone and reduce its battery

- reach-destination: Complete the mission when the destination is reached

- detect-gps-loss: Detect a GPS failure during flight

- detect-communication-loss: Detect a communication failure during flight

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

- controller-order-return: Follow the controller's return command

- return-to-source: Return the drone safely to the source

- controller-order-hover: Follow the controller's hover command

- hover: Keep the drone hovering while enough battery remains

- initiate-low-battery-return: Start emergency return when battery becomes low

- sacrifice-drone: Emergency landing when the controller orders sacrifice

3.2 Forest and area coverage

Source folder: plans/Search/. Domain name: forest-drone-search.

The search mission divides work among drones rather than giving the fleet one identical route. The operator marks a base and search polygon. engine/search_problem.py prepares the geographic search instance and drone assignments. The service requires at least two drones.

The normal flow checks battery and separation, moves to assigned locations, starts searching, marks areas covered, completes the forest search and returns drones to base. move-search requires a safe route, healthy systems and separation-ok. Its effects clear battery-checked and separation-ok so those conditions must be established again. The sample problem specifies a 10 m minimum separation between the two reference drones and a 50 m coverage width.

The final goal requires forest-search-completed, completed drone missions and both drones at base. The completion action in the domain explicitly accepts two distinct covered areas. It should not be interpreted as a universal proof that every arbitrary polygon cell has been visited: the generated problem, assignments and extracted routes also determine coverage. Separation predicates describe state checks and do not implement a peer radio protocol. No fixed communication direction count is specified.

Action catalogue (source order; this is not a newly solved plan):

- check-battery: Check if the drone has enough battery for a move

- notify-insufficient-battery: Record that the drone does not have enough battery

- check-drone-separation: Check the required distance between two drones

- start-search: Start searching an assigned forest area

- cover-area: Mark the assigned forest area as searched

- move-search: Move the drone to the next search location

- complete-forest-search: Mark the forest search as complete

- return-to-base: Return the drone to the base after searching

- complete-drone-mission: Complete the drone mission after returning to base

- emergency-low-battery-return: Start returning when the battery is too low

- detect-gps-loss: Detect GPS failure during flight

- detect-communication-loss: Detect communication failure during flight

- detect-health-failure: Detect when drone health becomes too low

3.3 V formation

Source folder: plans/vformation/. Domain name: v-formation-drone-mission.

A leader occupies the apex and equal ranks extend along the left and right wings. The expansion code requires an odd fleet count of at least three, giving $N = 1 + 2r$ for r ranks per wing. The geometry uses approximately 17.32 m behind and 10 m sideways per rank, as defined in the service constants and template.

Drones complete pre-flight checks, take off, climb and occupy assigned slots. The leader can hold while the launch counter advances. establish-v-formation requires the members to be ready. Each formation-cruise moves the leader and opens the formation; close-up-wings updates followers and restores the closed condition before the next leader leg.

The goal requires the established V, members in formation at cruise altitude, the destination reference location and completed missions. Neighbour relationships form two wing chains joined at the leader. The leader has two wing neighbours; an internal wing drone has inward and outward neighbours; a wing tip has one. Compass labels describe slot geometry and do not mean that every drone communicates in all eight compass directions.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the source location

- climb-to-cruise-altitude: Leader waits while the wing drones prepare for takeoff

- hold-position: Increase the launch delay by one second

- take-formation-slot: Move the drone into its assigned formation slot

- establish-v-formation: Establish the V formation when all drones are ready

- formation-cruise: Move the apex to the next location

- close-up-wings: Move all wing drones to the apex location

- complete-formation-mission: Complete the mission when the whole formation reaches the destination

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect neighbor communication failure

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

3.4 Two-dimensional grid formation

Source folder: plans/gridformation/. Domain name: grid-formation-drone-mission.

Drones occupy a horizontal rectangular grid with 10 m spacing. A perfect-square drone count gives a square. Other counts use an exact rectangular factorisation chosen by grid_dimensions. The GUI/service uses at least four drones for a visible grid; the lower-level expansion function supports smaller reference cases.

After pre-flight, takeoff and climb, each drone takes a grid slot. establish-grid-formation confirms readiness. The leader advances using formation-cruise, followed by close-up-grid for the members. The closed-state gate prevents another symbolic leader leg before the members have caught up.

The goal requires the established grid, member readiness, arrival and completed missions. A drone has up to four neighbours: front, back, left and right in the grid frame. Diagonals are excluded. Corner drones in a full rectangle normally have two neighbours, non-corner edge drones three and interior drones four. The generated facts are symmetric. Actual neighbour health is represented by neighbor-comm-ok rather than by a packet exchange inside PDDL.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the source

- climb-to-cruise-altitude: Climb to the cruise altitude

- hold-position: Leader waits for the other drones

- take-grid-slot: Move into the assigned grid slot

- establish-grid-formation: Establish the grid formation

- formation-cruise: Move the formation to the next location

- close-up-grid: Move grid members back to the leader

- complete-grid-mission: Complete the mission at the destination

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect neighbor communication failure

- detect-health-failure: Detect health failure

- notify-ground-controller: Notify the ground controller

3.5 Three-dimensional grid or cube formation

Source folder: plans/3dgridformation/. Domain name: cube-grid-formation-drone-mission.

This mission extends a grid into stacked horizontal layers. Eight drones form a 2 by 2 by 2 cube. Larger counts use the block dimensions calculated by cube_dimensions, with a partial layer where needed. The GUI/service requires at least eight drones, although the static PDDL template contains only a leader and one reference member for expansion.

The model adds layer objects, in-layer, layer-below, layer-launched and cleared-to-launch. The bottom layer starts cleared. open-next-layer releases a higher layer only after every drone in the lower layer has reached cruise altitude. Each drone has a layer-dependent altitude and vertical offset. After all layers are available, drones take cube slots, establish the cube, cruise and close up before the next leg.

The communication model permits up to 24 neighbour offsets: eight in the same layer, eight in the layer above and eight in the layer below. The two positions directly overhead and underneath are excluded. This is the actual repository rule, rather than the conventional 26-neighbour cubic mesh. Boundary and unfilled cells reduce the number of real neighbours. Layered launch is a symbolic ordering rule; physical clearance still depends on the generated trajectories and execution.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the source. Two gates, both about keeping aircraft out of each other's way while the stack is being built: - the leader goes first, everyone else waits out the launch delay; - no drone may leave the ground until every layer below its own has finished climbing (see `clear-next-layer`).

- climb-to-cruise-altitude: Climb to this drone's OWN cruise altitude - one per layer, so the stack separates vertically as it forms.

- hold-position: Leader waits for the other drones

- open-next-layer: Release the next layer up. Only applicable once every drone in the layer below has reached its cruise altitude, so exactly one layer is ever climbing at a time and no drone ever ascends through another's slot. This is the vertical counterpart of the staggered horizontal launch, and the second of the three collision-avoidance mechanisms in the header note. The conditional effect hands the launch clearance to every drone in the newly opened layer at once, which is what `takeoff` then checks.

- take-cube-slot: Move into the assigned cube slot

- establish-cube-formation: Establish the cube formation

- formation-cruise: Move the formation to the next location

- close-up-cube: Move every cube member up to the leader's new location. All-or-nothing: every member's battery and links are checked before any of them moves, so the block never partially advances.

- complete-cube-mission: Complete the mission at the destination

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure (the link to the ground controller)

- detect-neighbor-link-loss: Detect neighbor communication failure (the links to the up-to-24 drones adjacent to this one in the cube)

- detect-health-failure: Detect health failure

- notify-ground-controller: Notify the ground controller

3.6 Parabola formation

Source folder: plans/parabolaformation/. Domain name: parabola-formation-drone-mission.

Exactly three drones form a vertical cross-section: a central vertex and two outer arms. The vertex cruises at 50 m and the arms at 60 m in the template. The service places each arm 10 m sideways from the vertex. The higher arms create an upward-opening three-point parabola when viewed across the flight direction.

The mission adds explicit establish-comm-link and confirm-swarm-comm actions. Directed link-up facts represent peer-link availability. Drones take their slots and establish the parabola. formation-cruise advances the vertex and hold-parabola-shape brings the arms into the corresponding state while checking the required links and readiness.

Each drone must maintain links with both of the other drones. There are three unordered pairs, or six directed relationships when both directions are listed. This is two peers per drone, not a general two-compass-direction radio model. The goal requires all three swarm-comm-ready conditions, their target altitudes, arrival in formation and completed missions. Neighbour-link failure can clear link-up in both directions for the affected relationships.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the drone's own pad at the source - the pads are already on the formation's line, one slot spacing apart, so the three drones can lift off together without crossing

- climb-to-cruise-altitude: Climb from hover height to the drone's own cruise altitude - 50 m for the vertex, 60 m for each arm

- establish-comm-link: Set up the link from one drone to a neighbor - both must be up at cruise altitude with their radios working, and close enough for the link to reach

- confirm-swarm-comm: A drone is ready to fly in the swarm once it has a link to every one of its neighbors

- take-parabola-slot: Move the drone into its assigned parabola slot

- establish-parabola-formation: Establish the parabola when all three drones are in their slots and talking to each other. The shape must open upward: each arm flies exactly arm-rise above the vertex (so both arms are level with each other), and no two drones are closer than the minimum separation

- formation-cruise: Move the vertex to the next location - the arms hold their slots either side of it

- hold-parabola-shape: Bring both arms level with the vertex again. Only possible while every link in the formation is still up - the arms hold their slots from the positions they share with the vertex and with each other

- complete-parabola-mission: Complete the mission when the whole parabola reaches the destination

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect loss of the drone-to-drone links - every link the drone has drops with it, so the shape can't be re-formed until it is restored

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

3.7 Figure-eight formation

Source folder: plans/figure8formation/. Domain name: figure8-formation-drone-mission.

The operator selects the base, left lobe end, middle crossing and right lobe end, then a lap count. engine/figure8_problem.py builds a curved figure-eight and base transit paths. Drones hold ordered slots along a shared path. The current geometry spreads them around the full loop at equal slot distances; a blanket statement that every slot is exactly 10 m apart would be inaccurate. Ten metres is the spatial clearance target used by the geometry checks.

The symbolic sequence performs pre-flight checks, takeoff, climb, slot assignment and formation establishment. fly-to-pattern enters the loop. formation-cruise and follow-leader advance the leader and followers. start-next-lap counts intermediate laps; finish-laps counts the last lap. return-to-base and a final follower update precede complete-figure8-mission.

The geometry checks the crossing and the shared route. It uses a crossing path-gap threshold of 10 times sqrt(2) metres, approximately 14.14 m, for a right-angle crossing, corresponding to a 10 m straight-line clearance under that construction. Additional empty slots may be introduced when a full set of occupied equal slots would cause a crossing conflict. The route generator includes an extra flown lap to fill and empty the spread while providing the requested fully spread laps. These details belong to geometric execution and should be distinguished from the symbolic lap counter.

Communication is with the previous and next drone in the train order, up to two distinct peers. The PDDL goal completes the mission back at base and at cruise altitude; it does not require landed. Landing is handled by the generated execution path. The runtime uses shared pace messages and local speed control rather than a demonstrated direct peer-to-peer packet network.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the drone's own slot on the 8 - every drone already starts the spacing apart, so no staggered launch is needed

- climb-to-cruise-altitude: Climb from hover height to cruise altitude

- take-train-slot: Move the drone into its assigned train slot

- establish-figure8-formation: Establish the figure-8 formation when all drones are ready. The equal spacing must never bring two drones to the middle crossing point together: their gap along the path there must be at least the crossing clearance (10 m apart in a straight line)

- fly-to-pattern: Fly the train from the base to the middle of the 8

- formation-cruise: Move the leader to the next waypoint of the 8

- follow-leader: Move every follower along behind the leader

- start-next-lap: Finish a lap at the crossing and start the next one from the same point - only while more laps remain after the one being finished

- finish-laps: Count the last lap as flown - the train is at the middle of the 8 and no more laps remain

- return-to-base: Fly the train from the middle of the 8 back to the base

- complete-figure8-mission: Complete the mission when the train has flown every lap and is back at the base

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect loss of the link to the front/back neighbor

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

3.8 Star and circle drone show

Source folder: plans/starformation/. Domain name: star-formation-drone-mission.

This mission performs a show around one selected centre. Drones take off from separate pads and form a circle. During each lap they form a star, return to a circle, spin the circle, form the star again, spin the star and return to the circle. land-show completes the show on the original pads.

The domain records shape and phase progress. engine/star_problem.py generates the separate timed tracks. Drones retain their order around the centre during shape changes. The geometry adapts the number of star points to fleet size and scales the show to meet its spacing checks. Its documented transition criterion is at least 90 percent of the nominal spacing, so it should not be reported as an unconditional 10 m minimum throughout every transition.

ring-neighbor links each drone to the previous and next member around the ordered ring. A normal ring of at least three drones gives two distinct peers per drone. Small reference templates may have fewer distinct peers. Timed schedules use a shared pace clock so shape changes and rotations occur together. The final goal requires the requested laps, completed missions, return to source and landed states.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the drone's pad

- climb-to-cruise-altitude: Climb from hover height to the show's altitude

- form-circle: Every drone spreads out from its pad onto the circle

- form-star: Lap, step 1: the circle changes into the star

- form-circle-in-lap: Lap, step 2: the star changes back into the circle

- spin-circle: Lap, step 3: the circle turns once round

- form-star-again: Lap, step 4: the circle changes into the star again

- spin-star: Lap, step 5: the whole star turns once round its centre

- close-lap: Lap, step 6: the star changes back into the circle - the lap is done

- land-show: Every lap flown: spread back in onto the pads and land

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect loss of the link to a neighbor round the circle

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

3.9 Planetary orbit formation

Source folder: plans/planets/. Domain name: planets-formation-drone-mission.

Drones represent planets around a selected sun centre. The implementation supports two to eight drones. Orbit k has radius 50 times (k + 1) metres, from Mercury at 50 m to Neptune at 400 m. Neighbouring orbits turn in opposite directions.

Pre-flight, takeoff and climb precede enter-orbits. fly-lap advances the show counter and land-planets finishes it. A lap is one revolution of the outermost drone. Timed geometry coordinates the tracks so the drones line up again at lap boundaries. These are designed show trajectories rather than a physical model of planetary gravity or true orbital periods.

orbit-neighbor forms an open chain by orbit index. The innermost and outermost drones have one adjacent-orbit peer, and intermediate drones have two. Inward and outward describe neighbouring orbit assignments, not constant geographic bearings between moving drones. The final goal requires the requested laps, completed missions, drones back at source and landed.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the drone's pad

- climb-to-cruise-altitude: Climb from hover height to the show's altitude

- enter-orbits: Every drone flies out from its pad onto its own orbit - only once every drone is up, and only if no two neighbouring orbits turn the same way

- fly-lap: One lap: the outermost drone goes once round its orbit, every inner drone round its own as often as fits, and all are back on their starting line together

- land-planets: Every lap flown: fly back in onto the pads and land

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-neighbor-link-loss: Detect loss of the link to a neighbouring orbit's drone

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

3.10 Road inspection

Source folder: plans/Inspect/. Domain name: road-inspection-drone-mission.

A single drone follows a road from a selected source to destination, inspects its segments, returns along the road and lands at source. engine/road_inspection.py obtains the road shape from an OSRM routing service. Source and destination must lie on usable connected roads. Failure to obtain a route is reported before execution.

The generated road segments replace the template's single reference segment. The normal sequence is pre-flight-check, takeoff, climb-to-cruise-altitude, inspect-segment for the road chain, finish-inspection, return-along-road and land-at-source. Segment counters represent inspection progress.

The goal requires road-inspected, drone1 at source, landed and mission-completed. There is no inter-drone neighbour network in this single-drone mission. GCS commands and returned status are the relevant communication paths. A symbolic inspected segment records task completion in the model; it does not establish that a real camera detected a road defect.

Action catalogue (source order; this is not a newly solved plan):

- pre-flight-check: Check the drone before takeoff

- takeoff: Take off from the source end of the road

- climb-to-cruise-altitude: Climb from hover height to inspection altitude

- inspect-segment: Fly along the next segment of the road, inspecting it

- finish-inspection: At the destination with every segment inspected: the road is done

- return-along-road: Fly back along the same road, one segment at a time

- land-at-source: Land back at the source

- detect-gps-loss: Detect GPS failure

- detect-communication-loss: Detect communication failure

- detect-health-failure: Detect when drone health becomes too low

- notify-ground-controller: Inform the ground controller about a failure

4. Communication and coordination

4.1 Two different meanings of direction

Communication direction can mean data flow or the position of a neighbour. Data flow between the GCS and a drone is bidirectional: commands go down to the drone, and telemetry, acknowledgements and heartbeats return to the GCS. A formation direction means an adjacent slot, such as front or left. It does not specify a radio antenna beam, a frequency, or the number of independent physical communication channels.

Table 4.1. Modelled neighbour relationships

Mission	Relationship	Maximum distinct peers per drone
Travel	No fixed peer topology	Not specified
Search	Pairwise separation checks	No fixed communication degree
V	Adjacent drones along the wing chains	2; wing tips have 1
Horizontal grid	Front, back, left, right	4
Cube	8 same layer + 8 above + 8 below	24
Parabola	Each drone linked to both others	2
Figure-eight	Previous and next in train order	2
Star	Previous and next around ring order	2
Planets	Adjacent inner and outer orbits	2; endpoints have 1
Inspection	Single drone	0 inter-drone peers

These are maximum logical degrees, not the number of peers guaranteed in every instance. A two-drone ring cannot provide two different peers to either drone. Edges, corners, incomplete layers and fleet size change the actual counts. The GCS link is a separate relationship and is not included in the peer counts.

For a complete R by C horizontal grid, the number of unordered neighbour pairs is R(C - 1) + C(R - 1). Symmetric PDDL facts produce twice that number of directed facts. A V wing tree and an open orbit chain each have N - 1 unordered links. A normal ring with N at least three has N links. The three-drone parabola has three pairs. The cube count must be computed from occupied cells using the implemented offsets; multiplying N by 24 would overcount boundary links and count each pair twice.

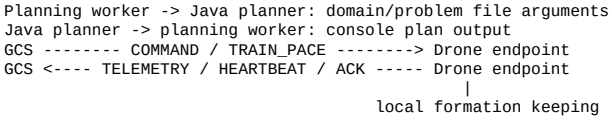
4.2 Symbolic link conditions

communication-ok represents general communication readiness, while neighbor-comm-ok represents a drone's neighbour communication condition in formation models. Facts such as grid-neighbor, wing-neighbor, train-neighbor, ring-neighbor and orbit-neighbor describe the intended topology. Most movement checks use the aggregate neighbor-comm-ok predicate rather than checking every pair individually. Merely listing neighbours therefore does not establish that a live network has verified those links.

The parabola model is more explicit: establish-comm-link creates link-up relationships, and confirmation checks required links. Even here, the facts are planning state. They are not UDP or MAVLink messages. notify-ground-controller records controller-notified symbolically and does not itself send a network packet. Runtime code must observe real state and implement the corresponding response.

4.3 Runtime data flow

Figure 4.1. Communication paths





engine/protocol.py defines a JSON-over-UDP transport with one datagram per message. The default GCS port is 14690, and a drone port is 14700 + SYSID; SYSID 1 therefore uses 14701. A command carries a drone identifier, command name, sequence and timestamp, with optional source, destination and parameters. Commands include NAVIGATE, ARM, TAKEOFF, HOLD, RESUME, RETURN_TO_LAUNCH, LAND and TRAIN_PACE. Telemetry carries position, velocity, battery, GPS, health and mission state. ACK reports acceptance or refusal; it does not prove that a flight operation has finished.

engine/drone_link.py separates socket handling from drone-thread logic. Its declared telemetry interval is 0.5 seconds and heartbeat interval is 1 second. These are configured intervals, not measured end-to-end delivery guarantees. UDP has no inherent reliable-delivery guarantee; protocol sequence fields and acknowledgements should not be described as proof of full reliable transport without examining the relevant retry and timeout behaviour.

The execution code also supports MAVLink mission construction and upload. engine/mavlink_mission.py, engine/mavlink_interpreter.py and services/mavlink_flight_service.py belong to that path. scripts/pddl_to_mavlink.py converts an already solved travel route into takeoff, waypoint and landing mission items and can validate them using a local flight-controller model or connect to a configured endpoint. The JSON protocol's ports should not be assumed to apply to every MAVLink or Renode path.

For trains and timed shows, services/thread_backend.py distributes a shared pace. engine/train_keeping.py compares each drone's own route progress with its required progress and calculates a local speed correction. The GCS can include the front drone's last known position and slow the common pace for a lagging drone. This is centrally distributed coordination with local control. The inspected code does not justify describing the PDDL neighbour graph as a fully implemented decentralised peer-to-peer radio network.

5. Safety, failures and model limitations

5.1 Energy and route safety

Movement usually requires adequate battery and healthy GPS, communication and drone state. Climb actions consume climb-energy, while follower updates often use cruise-leg-energy. The leader can instead consume edge-specific energy-required. These are intentionally different abstractions. Their values need calibration before they can support claims about real endurance.

The presence of minimum-return-battery does not mean every move reserves enough energy for a complete return. Several actions only compare battery with the next action cost. Similarly, the presence of max-battery does not impose a bound unless an action actually checks it. Route-level feasibility must be assessed from the complete action sequence and the required recovery route.

Problem retargeting can rotate, scale and translate the reference route. It recalculates edge distances and energy, preserving edge-specific reference cost rates. Restricted-area handling uses geometric checks and buffered detours. Formation services increase the corridor buffer to account for lateral formation reach. A safe leader corridor alone is insufficient evidence that every member trajectory is clear under every shape or turn.

5.2 Failure conditions and responses

The models include detection actions for GPS loss, communication loss and low health. Formation domains generally add neighbour-link loss. Effects can set failure flags and remove flying or in-formation. Travel includes controller orders, hovering, return and sacrifice state. Search includes an emergency low-battery return. The exact actions available in each mission are listed in Chapter 3 and defined in Appendix A.

These actions do not occur automatically in response to real time. A PDDL planner operates on a supplied state. An external event must first be reflected in that state before a detection action can apply. In particular, healthy initial facts will not become false merely because a runtime sensor later fails. A monitoring and replanning integration is needed to connect changing telemetry with updated planning facts.

A failure flag may also make the original success goal unreachable. A robust operational design should distinguish normal completion from safe recovery goals. A symbolic controller notification during communication loss is a model abstraction; actual delivery cannot be assumed when the link is unavailable.

5.3 Limits of the current representation

The models use instantaneous actions and discrete numeric updates, so they do not prove continuous collision avoidance, realistic acceleration limits or timing guarantees. A launch counter increment models ordering, not elapsed physical time. Separation checks at symbolic states are not continuous trajectory certificates. Detailed geometry and runtime control add necessary behaviour outside PDDL.

The report records source behaviour without silently changing it. Two points deserve further model review: the travel domain uses disjunction in some actions without listing disjunctive-preconditions, and the search domain uses equality without listing the corresponding requirement declaration. Planner acceptance of a file does not establish strict portability across PDDL tools. These observations do not constitute a newly executed planner failure.

Static templates are intentionally smaller than runtime missions in several folders. A two-drone cube template is an expansion seed, not the operational minimum cube size. Some template values and module comments describe earlier configurations. Current function behaviour and generated problem files should be checked when reporting exact fleet sizes or geometry.

6. Execution workflow and evaluation method

6.1 Reproducing a planning run

Install the project dependencies from requirements.txt and provide a usable Java installation and ENHSP JAR. Start the application using python main.py in the project environment. Select a mission and the required number of drones. Choose the appropriate map inputs: two endpoints for transit formations, a polygon for search, four points for the figure-eight, a centre for a show, or road endpoints for inspection.

Request planning and inspect the resulting route before execution. Preserve the generated problem.pddl and plan.txt together because the template alone does not describe the selected coordinates or expanded fleet. On success, inspect extracted routes, member offsets and timing schedules. On failure, retain the planner output and distinguish a timeout or missing dependency from a genuine infeasible problem. The wrapper's PlanNotFound exception is raised when no action sequence is parsed, so that exception name alone is not a complete mathematical diagnosis of unsolvability.

A direct template-solving command has this shape:

```
java -jar tools/enhsp/enhsp-dist/enhsp.jar -o plans/travell/domain.pddl -f plans/travell/problem.pddl
```

This solves the stored sample instance. To reproduce a GUI run, substitute the generated problem file under that run's timestamp directory. Some larger formation calls use mission-specific planner options, including naive grounding, so the exact service invocation matters when reproducing performance.

6.2 Suggested evaluation cases

Table 6.1. Validation cases to execute and record

Case	Expected property	Evidence to preserve
Normal travel	Goal reached through allowed edges	Generated problem and action sequence
Restricted-area transit	Member routes clear buffered region	Route geometry and clearance calculation
Insufficient battery	Unsafe movement cannot satisfy its gate	Initial values, failed preconditions or planner output
Search coverage	Required assignments covered and drones return	Coverage facts and full extracted routes
V and horizontal grid	Every member closes up before next leg	Ordered actions and neighbour facts
Cube ascent	Lower layer ready before next layer release	open-next-layer preconditions and action order
Parabola links	Every required peer link confirmed	link-up and swarm-comm-ready transitions
Figure-eight capacity	Unsafe layout rejected or safely adjusted	Slot count, crossing and shared-path clearance checks
Star and planets	Shared phase timing and final landing	Timed tracks, lap counters and final state
Road inspection	All generated segments processed, return and landing	Road geometry, segment counters and final goal
Runtime fault	Monitoring detects fault and chooses recovery	Telemetry, commands and any replanning problem

These are proposed evaluation cases, not claims that the checks passed. For formal evaluation, record drone count, waypoint count, action count, planner options, elapsed planning time, energy values, minimum geometric separation and final goal status. Use an independent plan validator where supported and replay the plan against the generated

initial state. Integration testing must additionally confirm that the executed route matches the planned route and that expected final flight states are reached.

6.3 Evidence available for this report

This documentation task inspected all ten current domain/problem pairs, the reference template and the relevant planning, geometry, protocol and execution source files. It did not run a new ten-mission planner benchmark or a flight experiment. Historical figures in plans/PLANS_STATUS.txt are dated and were not reused as current success rates. The complete source appendix provides an auditable snapshot of the planning models used for this report.

7. Conclusions and future work

The planning layer provides a common method for describing ten different drone missions. Domain actions state when movement, formation changes and task completion are permitted. Problem generation connects these rules to selected map locations and fleet sizes. ENHSP produces discrete plans, while the simulator converts them into routes and coordinated execution.

The communication design has two levels. PDDL describes readiness and intended neighbour relationships. Runtime code carries commands, telemetry and shared pace information. Clear separation of these levels makes the planning contribution easier to explain and prevents unsupported claims about decentralised networking or flight safety.

Further work can connect live telemetry to replanning states, express explicit recovery goals, validate return-energy reserves, improve PDDL requirement declarations, calibrate energy and timing models, and test member trajectories under faults and turns. These are extensions for evaluation and implementation, rather than results already established by the current report.

References

[R1] Reference Doc.pdf, supplied thesis-format example in the repository root. Used for report structure and front-matter organisation.

[R2] plans/travell/, plans/Search/, plans/vformation/, plans/gridformation/, plans/3dgridformation/, plans/parabolaformation/, plans/figure8formation/, plans/starformation/, plans/planets/ and plans/Inspect/: current domain and problem files, reproduced in Appendix A.

[R3] services/plan_service.py and engine/pddl_planner.py: plan discovery, instance preparation, planner invocation, output parsing and saved runs.

[R4] engine/pddl_problem.py, engine/search_problem.py, engine/figure8_problem.py, engine/star_problem.py, engine/planets_problem.py and engine/road_inspection.py: geographic preparation and formation geometry.

[R5] engine/protocol.py, engine/drone_link.py, services/thread_backend.py and engine/train_keeping.py: transport, messages and coordination.

[R6] engine/mavlink_mission.py, engine/mavlink_interpreter.py, services/mavlink_flight_service.py and scripts/pddl_to_mavlink.py: flight mission construction and execution integration.

[R7] README.md and plans/PLANS_STATUS.txt: repository overview and historical status context. Historical counts are not new measurements.

Appendix A. Complete PDDL source listings

The following listings reproduce all twenty current template files verbatim. They are domain and problem definitions, not newly solved action traces. Generated run instances are excluded because they depend on individual map selections and are stored separately. Each mission's domain contains the exact action parameters, preconditions and effects; its problem contains the initial values and goal. The explanations in Chapter 3 should be read together with these definitions.

A.1 Point-to-point travel

plans/travell/domain.pddl

```
(define (domain swarm-drone-mission)
    (:requirements
        :strips ;;Basic Planing Preconditions → Action → Effects
        :typing ;; Allows you to specify types drone, location, waypoint, controller
        :numeric-fluents ;; Allows numeric values such as: battery, energy-required, distance, health
        :negative-preconditions ;; Allows you to specify negative preconditions such as: (not (at ?d ?from))
        :conditional-effects ;; Allows you to specify conditional effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        waypoint
        controller
    )
    ;; Facts that describe the drone and mission state
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Mission locations
        (source ?l - location)
        (destination ?l - location)
        ;; Waypoint information
        (waypoint-location ?w - waypoint ?l - location)
        ;; Route information
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        (restricted-route ?from - location ?to - location)
        ;; Drone state
        (flying ?d - drone)
        (hovering ?d - drone)
        (mission-completed ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
        (drone-healthy ?d - drone)
        ;; Failure states
        (gps-lost ?d - drone)
        (communication-lost ?d - drone)
        (health-failure ?d - drone)
        ;; Controller commands and status
        (controller-notified ?d - drone)
        (controller-return ?d - drone)
        (controller-hover ?d - drone)
        (controller-sacrifice ?d - drone)
        ;; Warning messages
        (battery-insufficient ?d - drone)
        (health-warning ?d - drone)
        (gps-warning ?d - drone)
        (communication-warning ?d - drone)
        ;; Emergency states
        (emergency-return ?d - drone)
        (sacrificed ?d - drone)
        ;; Restricted area
        (no-fly-zone ?w - waypoint)
        ;; Mission checks
        (route-planned ?d - drone)
        (distance-checked ?d - drone)
        (battery-checked ?d - drone)
    )
    ;; Numeric values used by the planner
    (:functions
        ;; Battery information
        (battery ?d - drone)
        (max-battery ?d - drone)
        ;; Energy needed for a route
        (energy-required ?from - location ?to - location)
        ;; Distance between two locations
        (distance ?from - location ?to - location)
        ;; Drone health
        (health ?d - drone)
        (minimum-health ?d - drone)
        ;; Battery needed for emergency return
        (minimum-return-battery ?d - drone)
        ;; Battery used while hovering
        (hover-energy ?d - drone)
        ;; Energy used by the current route
        (route-energy ?d - drone)
        ;; Location coordinates
        (latitude ?l - location)
```

```
    (longitude ?l - location)
)
;; Check the distance before planning the route
(:action check-distance
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (connected ?from ?to)
        )
    :effect
        (distance-checked ?d)
)
;; Check whether the drone has enough battery
(:action check-battery
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (distance-checked ?d)
            (> (battery ?d)
               (energy-required ?from ?to))
        )
    :effect
        (battery-checked ?d)
)
;; Record that the battery is not enough
(:action notify-insufficient-battery
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (distance-checked ?d)
            (<= (battery ?d)
                (energy-required ?from ?to))
        )
    :effect
        (battery-insufficient ?d)
)
;; Create a route only when all safety checks pass
(:action plan-safe-route
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (battery-checked ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (route-planned ?d)
)
;; Move the drone and reduce its battery
(:action travel
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (route-planned ?d)
            (safe-route ?from ?to)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d)
               (energy-required ?from ?to))
        )
    :effect
        (and
            ;; Move drone to the new location
            (at ?d ?to)
            (not (at ?d ?from))
            ;; Drone is now flying
            (flying ?d)
            ;; Reduce battery by route energy
            (decrease
                (battery ?d)
                (energy-required ?from ?to)
            )
            ;; Route must be planned again for the next move
            (not (route-planned ?d))
        )
)
;; Complete the mission when the destination is reached
(:action reach-destination
```

```
    :parameters
        (?d - drone
         ?dest - location)
    :precondition
        (and
            (at ?d ?dest)
            (destination ?dest)
        )
    :effect
        (and
            (mission-completed ?d)
            (not (flying ?d))
        )
)
;; Detect a GPS failure during flight
(:action detect-gps-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (gps-ok ?d))
        )
    :effect
        (and
            (gps-lost ?d)
            (gps-warning ?d)
            (not (flying ?d))
        )
)
;; Detect a communication failure during flight
(:action detect-communication-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (communication-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (communication-warning ?d)
            (not (flying ?d))
        )
)
;; Detect when drone health becomes too low
(:action detect-health-failure
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (< (health ?d)
               (minimum-health ?d))
        )
    :effect
        (and
            (health-failure ?d)
            (health-warning ?d)
            (not (flying ?d))
        )
)
;; Inform the ground controller about a failure
(:action notify-ground-controller
    :parameters
        (?d - drone
         ?c - controller)
    :precondition
        (or
            (gps-lost ?d)
            (communication-lost ?d)
            (health-failure ?d)
        )
    :effect
        (controller-notified ?d)
)
;; Follow the controller's return command
(:action controller-order-return
    :parameters
        (?d - drone)
    :precondition
        (and
            (controller-notified ?d)
            (controller-return ?d)
        )
    :effect
        (emergency-return ?d)
)
;; Return the drone safely to the source
(:action return-to-source
    :parameters
        (?d - drone
         ?current - location
         ?src - location)
    :precondition
        (and
            (at ?d ?current)
            (source ?src)
            (emergency-return ?d)
            (safe-route ?current ?src)
            (> (battery ?d)
```

```
                (energy-required ?current ?src))
            )
        :effect
            (and
                (at ?d ?src)
                (not (at ?d ?current))
                (not (emergency-return ?d))
                (decrease
                    (battery ?d)
                    (energy-required ?current ?src)
                )
            )
    )
    ;; Follow the controller's hover command
    (:action controller-order-hover
        :parameters
            (?d - drone)
        :precondition
            (and
                (controller-notified ?d)
                (controller-hover ?d)
            )
        :effect
            (hovering ?d)
    )
    ;; Keep the drone hovering while enough battery remains
    (:action hover
        :parameters
            (?d - drone)
        :precondition
            (and
                (hovering ?d)
                (> (battery ?d)
                   (minimum-return-battery ?d))
            )
        :effect
            (decrease
                (battery ?d)
                (hover-energy ?d)
            )
    )
    ;; Start emergency return when battery becomes low
    (:action initiate-low-battery-return
        :parameters
            (?d - drone)
        :precondition
            (and
                (hovering ?d)
                (<= (battery ?d)
                    (minimum-return-battery ?d))
            )
        :effect
            (and
                (emergency-return ?d)
                (not (hovering ?d))
            )
    )
    ;; Emergency landing when the controller orders sacrifice
    (:action sacrifice-drone
        :parameters
            (?d - drone)
        :precondition
            (and
                (controller-notified ?d)
                (controller-sacrifice ?d)
            )
        :effect
            (and
                (sacrificed ?d)
                (not (flying ?d))
                (not (hovering ?d))
            )
    )
)
```

plans/travell/problem.pddl

```
(define (problem drone-mission-001)
    (:domain swarm-drone-mission)
    ;; Objects used in the mission
    (:objects
        drone1 - drone
        ground-controller - controller
        source - location
        destination - location
        waypoint1 - location
        waypoint2 - location
    )
    (:init
        ;; Drone starts at the source
        (at drone1 source)
        (source source)
        (destination destination)
        ;; Locations connected by possible routes
        (connected source destination)
        (connected source waypoint1)
        (connected waypoint1 waypoint2)
        (connected waypoint2 destination)
        (connected waypoint1 source)
        (connected waypoint2 waypoint1)
        (connected destination waypoint2)
```

```
        ;; Safe routes avoid the restricted direct route
        (safe-route source waypoint1)
        (safe-route waypoint1 waypoint2)
        (safe-route waypoint2 destination)
        (safe-route waypoint1 source)
        (safe-route waypoint2 waypoint1)
        (safe-route destination waypoint2)
        ;; Direct route is restricted
        (restricted-route source destination)
        ;; Drone starts in a normal operating state
        (gps-ok drone1)
        (communication-ok drone1)
        (drone-healthy drone1)
        ;; Geographic coordinates
        (= (latitude source) 13.0827)
        (= (longitude source) 80.2707)
        (= (latitude destination) 13.0674)
        (= (longitude destination) 80.2376)
        (= (latitude waypoint1) 13.0830)
        (= (longitude waypoint1) 80.2250)
        (= (latitude waypoint2) 13.0650)
        (= (longitude waypoint2) 80.2250)
        ;; Distances are provided by the mission-generation program
        (= (distance source destination) 4300)
        (= (distance source waypoint1) 3900)
        (= (distance waypoint1 waypoint2) 2100)
        (= (distance waypoint2 destination) 1800)
        (= (distance waypoint1 source) 3900)
        (= (distance waypoint2 waypoint1) 2100)
        (= (distance destination waypoint2) 1800)
        ;; Energy required for each route
        (= (energy-required source destination) 45)
        (= (energy-required source waypoint1) 20)
        (= (energy-required waypoint1 waypoint2) 12)
        (= (energy-required waypoint2 destination) 10)
        (= (energy-required waypoint1 source) 20)
        (= (energy-required waypoint2 waypoint1) 12)
        (= (energy-required destination waypoint2) 10)
        ;; Initial battery and battery limits
        (= (battery drone1) 85)
        (= (max-battery drone1) 100)
        ;; Start emergency return at or below 30% battery
        (= (minimum-return-battery drone1) 30)
        ;; Battery used for each hover action
        (= (hover-energy drone1) 2)
        ;; Initial drone health and minimum health
        (= (health drone1) 95)
        (= (minimum-health drone1) 40)
        ;; Waypoint 1 is inside the restricted area
        (no-fly-zone waypoint1)
    )
    (:goal
        (and
            ;; Mission is complete when the drone reaches the destination
            (mission-completed drone1)
            (at drone1 destination)
        )
    )
)
```

A.2 Forest and area coverage

plans/Search/domain.pddl

```
(define (domain forest-drone-search)
    (:requirements
        :strips
        :typing
        :numeric-fluents
        :negative-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        forest-area
        controller
    )
    ;; Facts that describe the drone and search mission
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Forest areas and their coverage
        (forest ?f - forest-area)
        (area-location ?a - forest-area ?l - location)
        (area-covered ?a - forest-area)
        ;; Drone-to-area assignment
        (assigned ?d - drone ?a - forest-area)
        ;; Location connectivity
        (connected ?from - location ?to - location)
        ;; Base location
        (is-base ?l - location)
        ;; Safe routes
        (safe-route ?from - location ?to - location)
        ;; Drone states
        (flying ?d - drone)
        (hovering ?d - drone)
```

```
    (searching ?d - drone)
    (search-completed ?d - drone)
    (returning ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    ;; Controller commands
    (controller-notified ?d - drone)
    (controller-return ?d - drone)
    (controller-hover ?d - drone)
    ;; Battery status
    (battery-insufficient ?d - drone)
    (battery-checked ?d - drone)
    ;; Collision avoidance
    (safe-separation ?d1 - drone ?d2 - drone)
    (separation-ok ?d - drone)
    (collision-warning ?d1 - drone ?d2 - drone)
    ;; Mission status
    (forest-search-completed)
    (mission-completed ?d - drone)
)
;; Numeric values used by the planner
(:functions
    ;; Battery information
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Energy and distance
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)
    ;; Drone size and separation
    (drone-width ?d - drone)
    (minimum-drone-separation ?d1 - drone ?d2 - drone)
    ;; Forest dimensions
    (forest-length ?f - forest-area)
    (forest-width ?f - forest-area)
    ;; Search coverage width
    (coverage-width ?d - drone)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Energy used while hovering
    (hover-energy ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
)
;; Check if the drone has enough battery for a move
(:action check-battery
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (connected ?from ?to)
            (> (battery ?d)
               (energy-required ?from ?to))
        )
    :effect
        (battery-checked ?d)
)
;; Record that the drone does not have enough battery
(:action notify-insufficient-battery
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (connected ?from ?to)
            (<= (battery ?d)
                (energy-required ?from ?to))
        )
    :effect
        (battery-insufficient ?d)
)
;; Check the required distance between two drones
(:action check-drone-separation
    :parameters
        (?d1 - drone
         ?d2 - drone
         ?l1 - location
         ?l2 - location)
    :precondition
        (and
            (at ?d1 ?l1)
            (at ?d2 ?l2)
            (not (= ?d1 ?d2))
            (>=
                (distance ?l1 ?l2)
                (minimum-drone-separation ?d1 ?d2)
            )
        )
    :effect
```

```
        (and
            (safe-separation ?d1 ?d2)
            (separation-ok ?d1)
            (separation-ok ?d2)
        )
)
;; Start searching an assigned forest area
(:action start-search
    :parameters
        (?d - drone
         ?a - forest-area
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (forest ?a)
            (area-location ?a ?l)
            (assigned ?d ?a)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (not (area-covered ?a))
        )
    :effect
        (and
            (searching ?d)
            (flying ?d)
        )
)
;; Mark the assigned forest area as searched
(:action cover-area
    :parameters
        (?d - drone
         ?a - forest-area
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (forest ?a)
            (area-location ?a ?l)
            (assigned ?d ?a)
            (searching ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (not (area-covered ?a))
        )
    :effect
        (and
            ;; Mark the forest area as covered
            (area-covered ?a)
            ;; Mark the drone's search as completed
            (search-completed ?d)
            (not (searching ?d))
        )
)
;; Move the drone to the next search location
(:action move-search
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (at ?d ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (separation-ok ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (battery-checked ?d)
            (> (battery ?d)
               (energy-required ?from ?to))
        )
    :effect
        (and
            ;; Move drone
            (at ?d ?to)
            (not (at ?d ?from))
            ;; Drone is flying
            (flying ?d)
            ;; Consume battery
            (decrease
                (battery ?d)
                (energy-required ?from ?to)
            )
            ;; Require fresh checks before the next move
            (not (battery-checked ?d))
            (not (separation-ok ?d))
        )
)
;; Mark the forest search as complete
(:action complete-forest-search
    :parameters
        (?a1 - forest-area
         ?a2 - forest-area)
    :precondition
        (and
            (area-covered ?a1)
            (area-covered ?a2)
            (not (= ?a1 ?a2))
```

```
        )
    :effect
        (forest-search-completed)
)
;; Return the drone to the base after searching
(:action return-to-base
    :parameters
        (?d - drone
         ?current - location
         ?base - location)
    :precondition
        (and
            (at ?d ?current)
            (is-base ?base)
            (connected ?current ?base)
            (safe-route ?current ?base)
            (separation-ok ?d)
            (forest-search-completed)
            (> (battery ?d)
               (energy-required ?current ?base))
        )
    :effect
        (and
            (at ?d ?base)
            (not (at ?d ?current))
            (not (flying ?d))
            (returning ?d)
            (decrease
                (battery ?d)
                (energy-required ?current ?base)
            )
        )
)
;; Complete the drone mission after returning to base
(:action complete-drone-mission
    :parameters
        (?d - drone
         ?base - location)
    :precondition
        (and
            (at ?d ?base)
            (returning ?d)
            (forest-search-completed)
        )
    :effect
        (and
            (mission-completed ?d)
            (not (returning ?d))
        )
)
;; Start returning when the battery is too low
(:action emergency-low-battery-return
    :parameters
        (?d - drone)
    :precondition
        (and
            (<=
                (battery ?d)
                (minimum-return-battery ?d)
            )
        )
    :effect
        (returning ?d)
)
;; Detect GPS failure during flight
(:action detect-gps-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (gps-ok ?d))
        )
    :effect
        (and
            (gps-lost ?d)
            (not (flying ?d))
        )
)
;; Detect communication failure during flight
(:action detect-communication-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (communication-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
        )
)
;; Detect when drone health becomes too low
(:action detect-health-failure
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (<
```

```
                    (health ?d)
                    (minimum-health ?d)
                )
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
            )
    )
)
```

plans/Search/SearchProblem.pddl

```
(define (problem forest-search-001)
    (:domain forest-drone-search)
    ;; Objects used in the search mission
    (:objects
        ;; Drones
        drone1 - drone
        drone2 - drone
        ;; Ground controller
        ground-controller - controller
        ;; One search area for each drone
        forest-lane1 - forest-area
        forest-lane2 - forest-area
        ;; Common starting and return location
        base - location
    )
    (:init
        ;; Forest search areas
        (forest forest-lane1)
        (forest forest-lane2)
        ;; Both drones start at the base
        (at drone1 base)
        (at drone2 base)
        (is-base base)
        ;; Assign one search area to each drone
        (assigned drone1 forest-lane1)
        (assigned drone2 forest-lane2)
        ;; Both drones start with normal system status
        (gps-ok drone1)
        (communication-ok drone1)
        (drone-healthy drone1)
        (gps-ok drone2)
        (communication-ok drone2)
        (drone-healthy drone2)
        ;; Initial battery
        (= (battery drone1) 100)
        (= (max-battery drone1) 100)
        (= (battery drone2) 100)
        (= (max-battery drone2) 100)
        ;; Battery level that triggers emergency return
        (= (minimum-return-battery drone1) 30)
        (= (minimum-return-battery drone2) 30)
        ;; Drone size and search coverage
        (= (drone-width drone1) 50)
        (= (drone-width drone2) 50)
        (= (coverage-width drone1) 50)
        (= (coverage-width drone2) 50)
        ;; Minimum distance required between the drones
        (= (minimum-drone-separation drone1 drone2) 10)
        (= (minimum-drone-separation drone2 drone1) 10)
        ;; Initial drone health
        (= (health drone1) 100)
        (= (health drone2) 100)
        (= (minimum-health drone1) 40)
        (= (minimum-health drone2) 40)
        ;; Battery used while hovering
        (= (hover-energy drone1) 2)
        (= (hover-energy drone2) 2)
        ;; Allow the first movement from the common base
        (separation-ok drone1)
        (separation-ok drone2)
        ;; Base coordinates
        (= (latitude base) 13.0827)
        (= (longitude base) 80.2707)
    )
    (:goal
        (and
            ;; Entire forest must be searched
            (forest-search-completed)
            ;; Both drone missions must be completed
            (mission-completed drone1)
            (mission-completed drone2)
            ;; Both drones must return to the base
            (at drone1 base)
            (at drone2 base)
        )
    )
)
```

A.3 V formation

plans/vformation/domain.pddl

```
(define (domain v-formation-drone-mission)
```

```
;; V-formation drone mission with staggered launch and coordinated flight
(:requirements
    :strips
    :typing
    :equality
    :numeric-fluents
    :negative-preconditions
    :disjunctive-preconditions
    :universal-preconditions
    :conditional-effects
)
;; Types used in the mission
(:types
    drone
    location
    controller
    direction
)
;; Facts that describe drone positions, states, and formation
(:predicates
    ;; Drone position
    (at ?d - drone ?l - location)
    ;; Current location of the whole formation
    (formation-at ?l - location)
    ;; Mission locations and routes
    (source ?l - location)
    (destination ?l - location)
    (connected ?from - location ?to - location)
    (safe-route ?from - location ?to - location)
    ;; Flight states
    (landed ?d - drone)
    (airborne ?d - drone)
    (flying ?d - drone)
    (at-cruise-altitude ?d - drone)
    ;; Formation slots
    (slot-apex ?d - drone)
    (slot-direction ?d - drone ?dir - direction)
    (has-slot ?d - drone)
    (wing-drone ?d - drone)
    (formation-leader ?d - drone)
    (in-formation ?d - drone)
    ;; Formation status
    (v-formation-established)
    (wings-closed)
    ;; Neighbor communication
    (wing-neighbor ?d1 - drone ?d2 - drone)
    (neighbor-comm-ok ?d - drone)
    ;; Pre-flight check
    (preflight-done ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
    (formation-mission-complete)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Drone altitude
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    ;; Energy consumption
    (climb-energy ?d - drone)
    (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    ;; Distance between locations
    (distance ?from - location ?to - location)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; V-formation slot offsets
    (slot-along-offset ?d - drone)
    (slot-cross-offset ?d - drone)
    ;; Staggered launch timing
    (seconds-since-leader-airborne)
    (wing-launch-delay)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (has-slot ?d)
```

```
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the source location
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            ;; Wings can take off after the leader waits
            (or
                (formation-leader ?d)
                (>= (seconds-since-leader-airborne) (wing-launch-delay))
            )
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Leader waits while the wing drones prepare for takeoff
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Increase the launch delay by one second
(:action hold-position
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (airborne ?lead)
            (< (seconds-since-leader-airborne) (wing-launch-delay))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
        )
    :effect
        (increase (seconds-since-leader-airborne) 1)
)
;; Move the drone into its assigned formation slot
(:action take-formation-slot
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (has-slot ?d)
            (at-cruise-altitude ?d)
            (>= (altitude ?d) (target-altitude ?d))
            (not (in-formation ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (in-formation ?d)
)
;; Establish the V formation when all drones are ready
(:action establish-v-formation
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (formation-leader ?lead)
            (slot-apex ?lead)
            (in-formation ?lead)
```

```
            (at ?lead ?l)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (neighbor-comm-ok ?lead)
            (forall (?w - drone)
                (or
                    (not (wing-drone ?w))
                    (and
                        (in-formation ?w)
                        (at ?w ?l)
                        (at-cruise-altitude ?w)
                        (>= (altitude ?w) (target-altitude ?w))
                        (neighbor-comm-ok ?w)
                    )
                )
            )
        )
    :effect
        (and
            (v-formation-established)
            (formation-at ?l)
            (wings-closed)
        )
)
;; Move the apex to the next location
(:action formation-cruise
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (v-formation-established)
            (wings-closed)
            (formation-leader ?lead)
            (slot-apex ?lead)
            (in-formation ?lead)
            (formation-at ?from)
            (at ?lead ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            ;; Wings must close up before the next leg
            (not (wings-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Move all wing drones to the apex location
(:action close-up-wings
    :parameters
        (?lead - drone
         ?to - location)
    :precondition
        (and
            (v-formation-established)
            (not (wings-closed))
            (formation-leader ?lead)
            (in-formation ?lead)
            (formation-at ?to)
            (at ?lead ?to)
            (forall (?w - drone)
                (or
                    (not (wing-drone ?w))
                    (and
                        (in-formation ?w)
                        (at-cruise-altitude ?w)
                        (>= (altitude ?w) (target-altitude ?w))
                        (gps-ok ?w)
                        (communication-ok ?w)
                        (drone-healthy ?w)
                        (neighbor-comm-ok ?w)
                        (> (battery ?w) (cruise-leg-energy ?w))
                    )
                )
            )
        )
    :effect
        (and
            (forall (?w - drone) (forall (?x - location)
                (when (and (wing-drone ?w) (at ?w ?x))
                    (not (at ?w ?x)))))
            (forall (?w - drone)
                (when (wing-drone ?w) (at ?w ?to)))
            (forall (?w - drone)
                (when (wing-drone ?w)
                    (decrease (battery ?w) (cruise-leg-energy ?w))))
            (wings-closed)
        )
)
```

```
;; Complete the mission when the whole formation reaches the destination
(:action complete-formation-mission
    :parameters
        (?lead - drone
         ?dest - location)
    :precondition
        (and
            (destination ?dest)
            (v-formation-established)
            (formation-at ?dest)
            (formation-leader ?lead)
            (slot-apex ?lead)
            (in-formation ?lead)
            (at ?lead ?dest)
            (forall (?w - drone)
                (or
                    (not (wing-drone ?w))
                    (and
                        (in-formation ?w)
                        (at ?w ?dest)
                    )
                )
            )
        )
    :effect
        (and
            (mission-completed ?lead)
            (forall (?w - drone)
                (when (wing-drone ?w)
                    (mission-completed ?w)))
            (formation-mission-complete)
        )
)
;; Detect GPS failure
(:action detect-gps-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (gps-ok ?d))
        )
    :effect
        (and
            (gps-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect communication failure
(:action detect-communication-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (communication-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect neighbor communication failure
(:action detect-neighbor-link-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (neighbor-comm-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect when drone health becomes too low
(:action detect-health-failure
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (< (health ?d) (minimum-health ?d))
        )
    :effect
        (and
            (health-failure ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Inform the ground controller about a failure
(:action notify-ground-controller
    :parameters
        (?d - drone
         ?c - controller)
```

```
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/vformation/problem.pddl

```
(define (problem v-formation-mission-001)
    (:domain v-formation-drone-mission)
    ;; Objects used in the V-formation mission
    (:objects
        drone-lead  - drone
        drone-left  - drone
        drone-right - drone
        ground-controller - controller
        source      - location
        destination - location
        ;; Possible compass directions for wing positions
        north       - direction
        north-east  - direction
        east        - direction
        south-east  - direction
        south       - direction
        south-west  - direction
        west        - direction
        north-west  - direction
    )
    (:init
        ;; All drones start landed at the source
        (at drone-lead  source)
        (at drone-left  source)
        (at drone-right source)
        (landed drone-lead)
        (landed drone-left)
        (landed drone-right)
        (= (altitude drone-lead)  0)
        (= (altitude drone-left)  0)
        (= (altitude drone-right) 0)
        (source source)
        (destination destination)
        ;; Direct route between source and destination
        (connected source destination)
        (connected destination source)
        (safe-route source destination)
        (safe-route destination source)
        ;; Assign the leader and two wing positions
        (formation-leader drone-lead)
        (slot-apex drone-lead)
        (slot-direction drone-left  north-west)
        (slot-direction drone-right south-west)
        (wing-drone drone-left)
        (wing-drone drone-right)
        (has-slot drone-lead)
        (has-slot drone-left)
        (has-slot drone-right)
        ;; V-formation slot offsets
        (= (slot-along-offset drone-lead)    0)
        (= (slot-cross-offset drone-lead)    0)
        (= (slot-along-offset drone-left)   -17.320508)
        (= (slot-cross-offset drone-left)   -10)
        (= (slot-along-offset drone-right)  -17.320508)
        (= (slot-cross-offset drone-right)   10)
        ;; Neighbor communication - each drone talks to its own left/right
        ;; neighbor along its wing (here, rank 1 either side of the leader)
        (wing-neighbor drone-lead drone-left)
        (wing-neighbor drone-left drone-lead)
        (wing-neighbor drone-lead drone-right)
        (wing-neighbor drone-right drone-lead)
        (neighbor-comm-ok drone-lead)
        (neighbor-comm-ok drone-left)
        (neighbor-comm-ok drone-right)
        ;; Leader waits 5 seconds before wing drones take off
        (= (seconds-since-leader-airborne) 0)
        (= (wing-launch-delay) 5)
        ;; All drones start with normal health and system status
        (gps-ok drone-lead)
        (communication-ok drone-lead)
        (drone-healthy drone-lead)
        (gps-ok drone-left)
        (communication-ok drone-left)
        (drone-healthy drone-left)
        (gps-ok drone-right)
        (communication-ok drone-right)
        (drone-healthy drone-right)
        (= (health drone-lead)  100)
        (= (health drone-left)  100)
        (= (health drone-right) 100)
        (= (minimum-health drone-lead)  40)
        (= (minimum-health drone-left)  40)
        (= (minimum-health drone-right) 40)
        ;; Initial battery
        (= (battery drone-lead)  100)
        (= (battery drone-left)  100)
```

```
        (= (battery drone-right) 100)
        (= (max-battery drone-lead)  100)
        (= (max-battery drone-left)  100)
        (= (max-battery drone-right) 100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-lead)  30)
        (= (minimum-return-battery drone-left)  30)
        (= (minimum-return-battery drone-right) 30)
        ;; Drones start at 2 m and climb to 50 m
        (= (hover-altitude drone-lead)  2)
        (= (hover-altitude drone-left)  2)
        (= (hover-altitude drone-right) 2)
        (= (target-altitude drone-lead)  50)
        (= (target-altitude drone-left)  50)
        (= (target-altitude drone-right) 50)
        ;; Battery used during the climb
        (= (climb-energy drone-lead)  4)
        (= (climb-energy drone-left) 4)
        (= (climb-energy drone-right) 4)
        ;; Battery used by each drone for one cruise leg
        (= (cruise-leg-energy drone-lead)  12)
        (= (cruise-leg-energy drone-left)  14)
        (= (cruise-leg-energy drone-right) 14)
        ;; Source and destination coordinates
        (= (latitude source)       18.3901547)
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
        ;; Direct route distance and energy cost
        (= (distance source destination) 3399.03)
        (= (distance destination source) 3399.03)
        (= (energy-required source destination) 10.00)
        (= (energy-required destination source) 10.00)
    )
    (:goal
        (and
            ;; V-formation is established
            (v-formation-established)
            (in-formation drone-lead)
            (in-formation drone-left)
            (in-formation drone-right)
            ;; All drones reached 50 m cruise altitude
            (at-cruise-altitude drone-lead)
            (at-cruise-altitude drone-left)
            (at-cruise-altitude drone-right)
            (>= (altitude drone-lead)  50)
            (>= (altitude drone-left)  50)
            (>= (altitude drone-right) 50)
            ;; Formation reached the destination
            (formation-at destination)
            (at drone-lead  destination)
            (at drone-left  destination)
            (at drone-right destination)
            ;; Mission is completed
            (formation-mission-complete)
            (mission-completed drone-lead)
            (mission-completed drone-left)
            (mission-completed drone-right)
        )
    )
)
```

A.4 Two-dimensional grid formation

plans/gridformation/domain.pddl

```
(define (domain grid-formation-drone-mission)
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Mission state and formation facts
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Current formation position
        (formation-at ?l - location)
        ;; Mission locations and routes
        (source ?l - location)
        (destination ?l - location)
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        ;; Flight state
        (landed ?d - drone)
        (airborne ?d - drone)
        (flying ?d - drone)
```

```
    (at-cruise-altitude ?d - drone)
    ;; Formation assignment
    (has-slot ?d - drone)
    (grid-member ?d - drone)
    (formation-leader ?d - drone)
    (in-formation ?d - drone)
    (grid-formation-established)
    (grid-closed)
    ;; Neighbor communication
    (grid-neighbor ?d1 - drone ?d2 - drone)
    (neighbor-comm-ok ?d - drone)
    ;; Pre-flight status
    (preflight-done ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
    (formation-mission-complete)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Altitude and energy
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    (climb-energy ?d - drone)
    (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    (distance ?from - location ?to - location)
    ;; Health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; Grid slot offsets
    (slot-along-offset ?d - drone)
    (slot-cross-offset ?d - drone)
    ;; Staggered launch timing
    (seconds-since-leader-airborne)
    (grid-launch-delay)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (has-slot ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the source
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (or
                (formation-leader ?d)
                (>= (seconds-since-leader-airborne) (grid-launch-delay))
            )
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb to the cruise altitude
(:action climb-to-cruise-altitude
```

```
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Leader waits for the other drones
(:action hold-position
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (airborne ?lead)
            (< (seconds-since-leader-airborne) (grid-launch-delay))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
        )
    :effect
        (increase (seconds-since-leader-airborne) 1)
)
;; Move into the assigned grid slot
(:action take-grid-slot
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (has-slot ?d)
            (at-cruise-altitude ?d)
            (>= (altitude ?d) (target-altitude ?d))
            (not (in-formation ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (in-formation ?d)
)
;; Establish the grid formation
(:action establish-grid-formation
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (formation-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?l)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (neighbor-comm-ok ?lead)
            (forall (?m - drone)
                (or
                    (not (grid-member ?m))
                    (and
                        (in-formation ?m)
                        (at ?m ?l)
                        (at-cruise-altitude ?m)
                        (>= (altitude ?m) (target-altitude ?m))
                        (neighbor-comm-ok ?m)
                    )
                )
            )
        )
    :effect
        (and
            (grid-formation-established)
            (formation-at ?l)
            (grid-closed)
        )
)
;; Move the formation to the next location
(:action formation-cruise
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (grid-formation-established)
            (grid-closed)
            (formation-leader ?lead)
            (in-formation ?lead)
            (formation-at ?from)
            (at ?lead ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
```

```
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            (not (grid-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Move grid members back to the leader
(:action close-up-grid
    :parameters
        (?lead - drone
         ?to - location)
    :precondition
        (and
            (grid-formation-established)
            (not (grid-closed))
            (formation-leader ?lead)
            (in-formation ?lead)
            (formation-at ?to)
            (at ?lead ?to)
            (forall (?m - drone)
                (or
                    (not (grid-member ?m))
                    (and
                        (in-formation ?m)
                        (at-cruise-altitude ?m)
                        (>= (altitude ?m) (target-altitude ?m))
                        (gps-ok ?m)
                        (communication-ok ?m)
                        (drone-healthy ?m)
                        (neighbor-comm-ok ?m)
                        (> (battery ?m) (cruise-leg-energy ?m))
                    )
                )
            )
        )
    :effect
        (and
            (forall (?m - drone) (forall (?x - location)
                (when
                    (and
                        (grid-member ?m)
                        (at ?m ?x)
                    )
                    (not (at ?m ?x))
                )
            ))
            (forall (?m - drone)
                (when
                    (grid-member ?m)
                    (at ?m ?to)
                )
            )
            (forall (?m - drone)
                (when
                    (grid-member ?m)
                    (decrease
                        (battery ?m)
                        (cruise-leg-energy ?m)
                    )
                )
            )
            (grid-closed)
        )
)
;; Complete the mission at the destination
(:action complete-grid-mission
    :parameters
        (?lead - drone
         ?dest - location)
    :precondition
        (and
            (destination ?dest)
            (grid-formation-established)
            (formation-at ?dest)
            (formation-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?dest)
            (forall (?m - drone)
                (or
                    (not (grid-member ?m))
                    (and
                        (in-formation ?m)
                        (at ?m ?dest)
                    )
                )
            )
        )
    :effect
        (and
            (mission-completed ?lead)
```

```
                (forall (?m - drone)
                    (when
                        (grid-member ?m)
                        (mission-completed ?m)
                    )
                )
                (formation-mission-complete)
            )
    )
    ;; Detect GPS failure
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect communication failure
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect neighbor communication failure
    (:action detect-neighbor-link-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (neighbor-comm-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect health failure
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Notify the ground controller
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/gridformation/problem.pddl

```
(define (problem grid-formation-mission-001)
    (:domain grid-formation-drone-mission)
    ;; Objects used in the mission
    (:objects
        drone-lead   - drone
        drone-member - drone
        ground-controller - controller
        source      - location
```

```
        destination - location
    )
    ;; Initial state
    (:init
        ;; Both drones start at the source
        (at drone-lead   source)
        (at drone-member source)
        (landed drone-lead)
        (landed drone-member)
        (= (altitude drone-lead)   0)
        (= (altitude drone-member) 0)
        (source source)
        (destination destination)
        ;; Direct route between source and destination
        (connected source destination)
        (connected destination source)
        (safe-route source destination)
        (safe-route destination source)
        ;; Assign formation leader and grid member
        (formation-leader drone-lead)
        (grid-member drone-member)
        (has-slot drone-lead)
        (has-slot drone-member)
        ;; Grid slot positions
        (= (slot-along-offset drone-lead)     0)
        (= (slot-cross-offset drone-lead)     0)
        (= (slot-along-offset drone-member)   0)
        (= (slot-cross-offset drone-member)  10)
        ;; Neighbor communication
        (grid-neighbor drone-lead drone-member)
        (grid-neighbor drone-member drone-lead)
        (neighbor-comm-ok drone-lead)
        (neighbor-comm-ok drone-member)
        ;; Staggered launch settings
        (= (seconds-since-leader-airborne) 0)
        (= (grid-launch-delay) 5)
        ;; System health starts normal
        (gps-ok drone-lead)
        (communication-ok drone-lead)
        (drone-healthy drone-lead)
        (gps-ok drone-member)
        (communication-ok drone-member)
        (drone-healthy drone-member)
        (= (health drone-lead)    100)
        (= (health drone-member)  100)
        (= (minimum-health drone-lead)    40)
        (= (minimum-health drone-member)  40)
        ;; Initial battery
        (= (battery drone-lead)    100)
        (= (battery drone-member)  100)
        (= (max-battery drone-lead)    100)
        (= (max-battery drone-member)  100)
        (= (minimum-return-battery drone-lead)    30)
        (= (minimum-return-battery drone-member)  30)
        ;; Altitude settings
        (= (hover-altitude drone-lead)    2)
        (= (hover-altitude drone-member)  2)
        (= (target-altitude drone-lead)    50)
        (= (target-altitude drone-member)  50)
        ;; Battery used during climb
        (= (climb-energy drone-lead)    4)
        (= (climb-energy drone-member)  4)
        ;; Battery used for each cruise leg
        (= (cruise-leg-energy drone-lead)    12)
        (= (cruise-leg-energy drone-member)  14)
        ;; Source and destination coordinates
        (= (latitude source)       18.3901547)
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
        ;; Direct route distance and energy cost
        (= (distance source destination) 3399.03)
        (= (distance destination source) 3399.03)
        (= (energy-required source destination) 10.00)
        (= (energy-required destination source) 10.00)
    )
    ;; Mission goal
    (:goal
        (and
            ;; Grid formation is established
            (grid-formation-established)
            (in-formation drone-lead)
            (in-formation drone-member)
            ;; Both drones reach cruise altitude
            (at-cruise-altitude drone-lead)
            (at-cruise-altitude drone-member)
            (>= (altitude drone-lead)    50)
            (>= (altitude drone-member)  50)
            ;; Formation reaches the destination
            (formation-at destination)
            (at drone-lead   destination)
            (at drone-member destination)
            ;; Mission is completed
            (formation-mission-complete)
            (mission-completed drone-lead)
            (mission-completed drone-member)
        )
    )
)
```

A.5 Three-dimensional grid or cube formation

plans/3dgridformation/domain.pddl

```
(define (domain cube-grid-formation-drone-mission)
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
        layer
    )
    ;; Mission state and formation facts
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Current formation position
        (formation-at ?l - location)
        ;; Mission locations and routes
        (source ?l - location)
        (destination ?l - location)
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        ;; Flight state
        (landed ?d - drone)
        (airborne ?d - drone)
        (flying ?d - drone)
        (at-cruise-altitude ?d - drone)
        ;; Formation assignment
        (has-slot ?d - drone)
        (cube-member ?d - drone)
        (formation-leader ?d - drone)
        (in-formation ?d - drone)
        (cube-formation-established)
        (cube-closed)
        (cube-neighbor ?d1 - drone ?d2 - drone) ;; The link to the up-to-24 drones adjacent to this one in the
cube
        (neighbor-comm-ok ?d - drone)     ;; The link to the up-to-24 drones adjacent to this one in the cube
        ;; Layer-by-layer ascent (see the header note)
        (in-layer ?d - drone ?l - layer)
        (layer-below ?lo - layer ?hi - layer)
        (layer-launched ?l - layer)
        (cleared-to-launch ?d - drone)
        ;; Pre-flight status
        (preflight-done ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
        (drone-healthy ?d - drone)
        ;; Failure states
        (gps-lost ?d - drone)
        (communication-lost ?d - drone)
        (health-failure ?d - drone)
        (controller-notified ?d - drone)
        ;; Mission status
        (mission-completed ?d - drone)
        (formation-mission-complete)
    )
    ;; Numeric values used by the planner
    (:functions
        ;; Battery
        (battery ?d - drone)
        (max-battery ?d - drone)
        (minimum-return-battery ?d - drone)
        ;; Altitude and energy
        (altitude ?d - drone)
        (hover-altitude ?d - drone)
        (target-altitude ?d - drone)
        (climb-energy ?d - drone)
        (cruise-leg-energy ?d - drone)
        (energy-required ?from - location ?to - location)
        (distance ?from - location ?to - location)
        ;; Health
        (health ?d - drone)
        (minimum-health ?d - drone)
        ;; Location coordinates
        (latitude ?l - location)
        (longitude ?l - location)
        (slot-along-offset ?d - drone) ;;along the direction of travel
        (slot-cross-offset ?d - drone) ;;perpendicular to the direction of travel
        (slot-layer ?d - drone)        ;;vertical layer in the cube formation
        (slot-lift-offset ?d - drone)  ;;vertical offset from the layer's base altitude
        ;; Staggered launch timing
        (seconds-since-leader-airborne)
        (cube-launch-delay)
    )
    ;; Check the drone before takeoff
    (:action pre-flight-check
        :parameters
```

```
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (has-slot ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the source.
;;
;; Two gates, both about keeping aircraft out of each other's way while
;; the stack is being built:
;;   - the leader goes first, everyone else waits out the launch delay;
;;   - no drone may leave the ground until every layer below its own has
;;     finished climbing (see `clear-next-layer`).
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (cleared-to-launch ?d)
            (or
                (formation-leader ?d)
                (>= (seconds-since-leader-airborne) (cube-launch-delay))
            )
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb to this drone's OWN cruise altitude - one per layer, so the
;; stack separates vertically as it forms.
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Leader waits for the other drones
(:action hold-position
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (airborne ?lead)
            (< (seconds-since-leader-airborne) (cube-launch-delay))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
        )
    :effect
        (increase (seconds-since-leader-airborne) 1)
)
;; Release the next layer up.
;;
;; Only applicable once every drone in the layer below has reached its
;; cruise altitude, so exactly one layer is ever climbing at a time and no
;; drone ever ascends through another's slot. This is the vertical
;; counterpart of the staggered horizontal launch, and the second of the
;; three collision-avoidance mechanisms in the header note.
;;
;; The conditional effect hands the launch clearance to every drone in the
;; newly opened layer at once, which is what `takeoff` then checks.
(:action open-next-layer
    :parameters
        (?lo - layer
```

```
         ?hi - layer)
    :precondition
        (and
            (layer-below ?lo ?hi)
            (layer-launched ?lo)
            (not (layer-launched ?hi))
            (forall (?d - drone)
                (or
                    (not (in-layer ?d ?lo))
                    (at-cruise-altitude ?d)
                )
            )
        )
    :effect
        (and
            (layer-launched ?hi)
            (forall (?d - drone)
                (when
                    (in-layer ?d ?hi)
                    (cleared-to-launch ?d)
                )
            )
        )
)
;; Move into the assigned cube slot
(:action take-cube-slot
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (has-slot ?d)
            (at-cruise-altitude ?d)
            (>= (altitude ?d) (target-altitude ?d))
            (not (in-formation ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (in-formation ?d)
)
;; Establish the cube formation
(:action establish-cube-formation
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (formation-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?l)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (neighbor-comm-ok ?lead)
            ;; Every layer has been released, so the whole stack is up.
            ;; NB the quantified variable must not be `?l` - that name is
            ;; already bound to this action's location parameter.
            (forall (?ly - layer) (layer-launched ?ly))
            (forall (?m - drone)
                (or
                    (not (cube-member ?m))
                    (and
                        (in-formation ?m)
                        (at ?m ?l)
                        (at-cruise-altitude ?m)
                        (>= (altitude ?m) (target-altitude ?m))
                        (neighbor-comm-ok ?m)
                    )
                )
            )
        )
    :effect
        (and
            (cube-formation-established)
            (formation-at ?l)
            (cube-closed)
        )
)
;; Move the formation to the next location
(:action formation-cruise
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (cube-formation-established)
            (cube-closed)
            (formation-leader ?lead)
            (in-formation ?lead)
            (formation-at ?from)
            (at ?lead ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
```

```
        )
    :effect
        (and
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            (not (cube-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Move every cube member up to the leader's new location.
;;
;; All-or-nothing: every member's battery and links are checked before
;; any of them moves, so the block never partially advances.
(:action close-up-cube
    :parameters
        (?lead - drone
         ?to - location)
    :precondition
        (and
            (cube-formation-established)
            (not (cube-closed))
            (formation-leader ?lead)
            (in-formation ?lead)
            (formation-at ?to)
            (at ?lead ?to)
            (forall (?m - drone)
                (or
                    (not (cube-member ?m))
                    (and
                        (in-formation ?m)
                        (at-cruise-altitude ?m)
                        (>= (altitude ?m) (target-altitude ?m))
                        (gps-ok ?m)
                        (communication-ok ?m)
                        (drone-healthy ?m)
                        (neighbor-comm-ok ?m)
                        (> (battery ?m) (cruise-leg-energy ?m))
                    )
                )
            )
        )
    :effect
        (and
            (forall (?m - drone) (forall (?x - location)
                (when
                    (and
                        (cube-member ?m)
                        (at ?m ?x)
                    )
                    (not (at ?m ?x))
                )
            ))
            (forall (?m - drone)
                (when
                    (cube-member ?m)
                    (at ?m ?to)
                )
            )
            (forall (?m - drone)
                (when
                    (cube-member ?m)
                    (decrease
                        (battery ?m)
                        (cruise-leg-energy ?m)
                    )
                )
            )
            (cube-closed)
        )
)
;; Complete the mission at the destination
(:action complete-cube-mission
    :parameters
        (?lead - drone
         ?dest - location)
    :precondition
        (and
            (destination ?dest)
            (cube-formation-established)
            (formation-at ?dest)
            (formation-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?dest)
            (forall (?m - drone)
                (or
                    (not (cube-member ?m))
                    (and
                        (in-formation ?m)
                        (at ?m ?dest)
                    )
                )
            )
        )
    :effect
        (and
            (mission-completed ?lead)
            (forall (?m - drone)
                (when
                    (cube-member ?m)
                    (mission-completed ?m)
```

```
                    )
                )
                (formation-mission-complete)
            )
    )
    ;; Detect GPS failure
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect communication failure (the link to the ground controller)
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect neighbor communication failure (the links to the up-to-24
    ;; drones adjacent to this one in the cube)
    (:action detect-neighbor-link-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (neighbor-comm-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Detect health failure
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Notify the ground controller
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/3dgridformation/problem.pddl

```
(define (problem cube-grid-formation-mission-001)
    (:domain cube-grid-formation-drone-mission)
    ;; This file is a TEMPLATE, not the mission that flies. It describes the
    ;; smallest possible case - the leader plus one member beside it, a
    ;; single-layer 1x1x2 "cube" - and `engine.pddl_problem.expand_cube_drones`
    ;; rebuilds it into whatever layers x rows x columns block the checked
    ;; drone count calls for, then `retarget_cube_problem` relocates it onto
    ;; the picked source/destination.
    ;;
    ;; FORMATTING RULE: keep every `(= (fluent object) value)` fact on ONE
    ;; LINE. The expansion machinery clones facts line by line
```

```
;; (`_clone_named_lines`), so a fact split across several lines is cloned
;; into a valueless fragment and the planner then reports the problem
;; unsolvable - exactly the defect that broke plans/gridformation/ on
;; 2026-09-16.
;; Objects used in the mission
(:objects
    drone-lead   - drone
    drone-member - drone
    ground-controller - controller
    source      - location
    destination - location
    ;; One object per horizontal layer, bottom first. The template is a
    ;; single layer; expand_cube_drones widens this to layer0..layerN-1.
    layer0 - layer
)
;; Initial state
(:init
    ;; Both drones start at the source
    (at drone-lead   source)
    (at drone-member source)
    (landed drone-lead)
    (landed drone-member)
    (= (altitude drone-lead)   0)
    (= (altitude drone-member) 0)
    (source source)
    (destination destination)
    ;; Direct route between source and destination
    (connected source destination)
    (connected destination source)
    (safe-route source destination)
    (safe-route destination source)
    ;; Assign formation leader and cube member
    (formation-leader drone-lead)
    (cube-member drone-member)
    (has-slot drone-lead)
    (has-slot drone-member)
    ;; Cube slot positions - horizontal pair, then the vertical pair.
    ;; The template is a single layer, so both drones sit at layer 0 with
    ;; no lift above the bottom layer's cruise altitude.
    (= (slot-along-offset drone-lead)     0)
    (= (slot-cross-offset drone-lead)     0)
    (= (slot-layer drone-lead)            0)
    (= (slot-lift-offset drone-lead)      0)
    (= (slot-along-offset drone-member)   0)
    (= (slot-cross-offset drone-member)  10)
    (= (slot-layer drone-member)          0)
    (= (slot-lift-offset drone-member)    0)
    ;; Neighbor communication. For this single-layer template the 24-way
    ;; relation collapses to one symmetric pair; `expand_cube_drones`
    ;; strips these and rebuilds the full 24-connected mesh for the real
    ;; block.
    (cube-neighbor drone-lead drone-member)
    (cube-neighbor drone-member drone-lead)
    (neighbor-comm-ok drone-lead) ;; communications are OK at the start of the mission
    (neighbor-comm-ok drone-member);; communication with 24 neighbors.
    ;; Layer-by-layer ascent. Both template drones sit in the bottom
    ;; layer, which starts already launched - so they are cleared to take
    ;; off immediately. A single-layer template has nothing above layer 0,
    ;; so there are no `layer-below` facts and `open-next-layer` never
    ;; fires; expand_cube_drones strips and rebuilds all four fact kinds
    ;; for the real block.
    (in-layer drone-lead   layer0)
    (in-layer drone-member layer0)
    (layer-launched layer0)
    (cleared-to-launch drone-lead)
    (cleared-to-launch drone-member)
    ;; Staggered launch settings
    (= (seconds-since-leader-airborne) 0)
    (= (cube-launch-delay) 5)
    ;; System health starts normal
    (gps-ok drone-lead)
    (communication-ok drone-lead)
    (drone-healthy drone-lead)
    (gps-ok drone-member)
    (communication-ok drone-member)
    (drone-healthy drone-member)
    (= (health drone-lead)    100)
    (= (health drone-member)  100)
    (= (minimum-health drone-lead)    40)
    (= (minimum-health drone-member)  40)
    ;; Initial battery
    (= (battery drone-lead)    100)
    (= (battery drone-member)  100)
    (= (max-battery drone-lead)    100)
    (= (max-battery drone-member)  100)
    (= (minimum-return-battery drone-lead)    30)
    (= (minimum-return-battery drone-member)  30)
    ;; Altitude settings. `target-altitude` is per drone because it is
    ;; per LAYER - expand_cube_drones rewrites it as
    ;; base + layer * layer-spacing for every cell.
    (= (hover-altitude drone-lead)    2)
    (= (hover-altitude drone-member)  2)
    (= (target-altitude drone-lead)    50)
    (= (target-altitude drone-member)  50)
    ;; Battery used during climb
    (= (climb-energy drone-lead)    4)
    (= (climb-energy drone-member)  4)
    ;; Battery used for each cruise leg
    (= (cruise-leg-energy drone-lead)    12)
    (= (cruise-leg-energy drone-member)  14)
    ;; Source and destination coordinates
    (= (latitude source)       18.3901547)
```

```
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
        ;; Direct route distance and energy cost
        (= (distance source destination) 3399.03)
        (= (distance destination source) 3399.03)
        (= (energy-required source destination) 10.00)
        (= (energy-required destination source) 10.00)
    )
    ;; Mission goal
    (:goal
        (and
            ;; Cube formation is established
            (cube-formation-established)
            (in-formation drone-lead)
            (in-formation drone-member)
            ;; Both drones reached their own layer's cruise altitude
            (at-cruise-altitude drone-lead)
            (at-cruise-altitude drone-member)
            ;; Formation reaches the destination
            (formation-at destination)
            (at drone-lead   destination)
            (at drone-member destination)
            ;; Mission is completed
            (formation-mission-complete)
            (mission-completed drone-lead)
            (mission-completed drone-member)
        )
    )
)
```

A.6 Parabola formation

plans/parabolaformation/domain.pddl

```
(define (domain parabola-formation-drone-mission)
    ;; Upward-parabola formation: three drones fly side by side on one line
    ;; across the direction of travel. The middle drone (the vertex) cruises
    ;; at 50 m and the two arm drones either side of it at 60 m, so seen
    ;; from behind the three make a "U" - an upward-opening parabola - and
    ;; fly it from the source to the destination. Every drone keeps a live
    ;; link with each of the other two the whole way
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Facts that describe drone positions, states, and formation
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Current location of the whole formation
        (formation-at ?l - location)
        ;; Mission locations and routes
        (source ?l - location)
        (destination ?l - location)
        (connected ?from - location ?to - location)
        (safe-route ?from - location ?to - location)
        ;; Flight states
        (landed ?d - drone)
        (airborne ?d - drone)
        (flying ?d - drone)
        (at-cruise-altitude ?d - drone)
        ;; Formation slots: the vertex in the middle (lowest point of the
        ;; parabola) and an arm drone either side of it on the same line
        (formation-leader ?d - drone)
        (slot-vertex ?d - drone)
        (arm-drone ?d - drone)
        (has-slot ?d - drone)
        (in-formation ?d - drone)
        ;; Formation status
        (parabola-established)
        (shape-held)
        ;; Communication between the drones. parabola-neighbor lists every
        ;; pair that must talk (all three pairs here - the two arms talk to
        ;; the vertex and to each other); link-up is a link that has been
        ;; set up and is in range; swarm-comm-ready means a drone has a
        ;; working link with every one of its neighbors
        (parabola-neighbor ?d1 - drone ?d2 - drone)
        (link-up ?d1 - drone ?d2 - drone)
        (neighbor-comm-ok ?d - drone)
        (swarm-comm-ready ?d - drone)
        ;; Pre-flight check
        (preflight-done ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
```

```
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
    (parabola-mission-complete)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Drone altitude
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    ;; Energy consumption
    (climb-energy ?d - drone)
    (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    ;; Distance between locations
    (distance ?from - location ?to - location)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; Parabola slot offsets: metres behind the vertex (negative =
    ;; behind) and metres to its right (negative = left). Both arms sit
    ;; level with the vertex, on the same line across the route
    (slot-along-offset ?d - drone)
    (slot-cross-offset ?d - drone)
    ;; Parabola shape: how much higher each arm flies than the vertex
    ;; (it must be exactly this, the same for both arms, so the shape is
    ;; a symmetric "U"), and the smallest 3-D gap any two drones may have
    (arm-rise)
    (min-separation)
    ;; Communication: straight-line distance between two drones in the
    ;; formation, and how far a drone-to-drone link can reach
    (link-distance ?d1 - drone ?d2 - drone)
    (comm-range)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (has-slot ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the drone's own pad at the source - the pads are
;; already on the formation's line, one slot spacing apart, so the
;; three drones can lift off together without crossing
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb from hover height to the drone's own cruise altitude - 50 m for
;; the vertex, 60 m for each arm
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
```

```
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Set up the link from one drone to a neighbor - both must be up at
;; cruise altitude with their radios working, and close enough for the
;; link to reach
(:action establish-comm-link
    :parameters
        (?d1 - drone
         ?d2 - drone)
    :precondition
        (and
            (parabola-neighbor ?d1 ?d2)
            (not (link-up ?d1 ?d2))
            (airborne ?d1)
            (airborne ?d2)
            (at-cruise-altitude ?d1)
            (at-cruise-altitude ?d2)
            (communication-ok ?d1)
            (communication-ok ?d2)
            (neighbor-comm-ok ?d1)
            (neighbor-comm-ok ?d2)
            (<= (link-distance ?d1 ?d2) (comm-range))
        )
    :effect
        (link-up ?d1 ?d2)
)
;; A drone is ready to fly in the swarm once it has a link to every one
;; of its neighbors
(:action confirm-swarm-comm
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (swarm-comm-ready ?d))
            (communication-ok ?d)
            (neighbor-comm-ok ?d)
            (forall (?n - drone)
                (or
                    (not (parabola-neighbor ?d ?n))
                    (link-up ?d ?n)
                )
            )
        )
    :effect
        (swarm-comm-ready ?d)
)
;; Move the drone into its assigned parabola slot
(:action take-parabola-slot
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (has-slot ?d)
            (at-cruise-altitude ?d)
            (>= (altitude ?d) (target-altitude ?d))
            (not (in-formation ?d))
            (swarm-comm-ready ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (in-formation ?d)
)
;; Establish the parabola when all three drones are in their slots and
;; talking to each other. The shape must open upward: each arm flies
;; exactly arm-rise above the vertex (so both arms are level with each
;; other), and no two drones are closer than the minimum separation
(:action establish-parabola-formation
    :parameters
        (?vtx - drone
         ?l - location)
    :precondition
        (and
            (formation-leader ?vtx)
            (slot-vertex ?vtx)
            (in-formation ?vtx)
            (at ?vtx ?l)
            (at-cruise-altitude ?vtx)
            (>= (altitude ?vtx) (target-altitude ?vtx))
            (swarm-comm-ready ?vtx)
            (forall (?a - drone)
                (or
                    (not (arm-drone ?a))
                    (and
                        (in-formation ?a)
                        (at ?a ?l)
                        (at-cruise-altitude ?a)
                        (>= (altitude ?a) (target-altitude ?a))
                        (= (- (target-altitude ?a) (target-altitude ?vtx)) (arm-rise))
```

```
                        (>= (link-distance ?vtx ?a) (min-separation))
                        (swarm-comm-ready ?a)
                    )
                )
            )
        )
    :effect
        (and
            (parabola-established)
            (formation-at ?l)
            (shape-held)
        )
)
;; Move the vertex to the next location - the arms hold their slots
;; either side of it
(:action formation-cruise
    :parameters
        (?vtx - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (parabola-established)
            (shape-held)
            (formation-leader ?vtx)
            (slot-vertex ?vtx)
            (in-formation ?vtx)
            (formation-at ?from)
            (at ?vtx ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (at-cruise-altitude ?vtx)
            (>= (altitude ?vtx) (target-altitude ?vtx))
            (gps-ok ?vtx)
            (communication-ok ?vtx)
            (drone-healthy ?vtx)
            (neighbor-comm-ok ?vtx)
            (swarm-comm-ready ?vtx)
            (> (battery ?vtx) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?vtx ?from))
            (at ?vtx ?to)
            ;; The arms must re-form the parabola before the next leg
            (not (shape-held))
            (decrease (battery ?vtx) (energy-required ?from ?to))
        )
)
;; Bring both arms level with the vertex again. Only possible while
;; every link in the formation is still up - the arms hold their slots
;; from the positions they share with the vertex and with each other
(:action hold-parabola-shape
    :parameters
        (?vtx - drone
         ?to - location)
    :precondition
        (and
            (parabola-established)
            (not (shape-held))
            (formation-leader ?vtx)
            (in-formation ?vtx)
            (formation-at ?to)
            (at ?vtx ?to)
            (forall (?d1 - drone)
                (forall (?d2 - drone)
                    (or
                        (not (parabola-neighbor ?d1 ?d2))
                        (link-up ?d1 ?d2)
                    )
                )
            )
            (forall (?a - drone)
                (or
                    (not (arm-drone ?a))
                    (and
                        (in-formation ?a)
                        (at-cruise-altitude ?a)
                        (>= (altitude ?a) (target-altitude ?a))
                        (gps-ok ?a)
                        (communication-ok ?a)
                        (drone-healthy ?a)
                        (neighbor-comm-ok ?a)
                        (swarm-comm-ready ?a)
                        (> (battery ?a) (cruise-leg-energy ?a))
                    )
                )
            )
        )
    :effect
        (and
            (forall (?a - drone) (forall (?x - location)
                (when (and (arm-drone ?a) (at ?a ?x))
                    (not (at ?a ?x)))))
            (forall (?a - drone)
                (when (arm-drone ?a) (at ?a ?to)))
            (forall (?a - drone)
                (when (arm-drone ?a)
                    (decrease (battery ?a) (cruise-leg-energy ?a))))
            (shape-held)
```

```
        )
)
;; Complete the mission when the whole parabola reaches the destination
(:action complete-parabola-mission
    :parameters
        (?vtx - drone
         ?dest - location)
    :precondition
        (and
            (destination ?dest)
            (parabola-established)
            (shape-held)
            (formation-at ?dest)
            (formation-leader ?vtx)
            (slot-vertex ?vtx)
            (in-formation ?vtx)
            (at ?vtx ?dest)
            (forall (?a - drone)
                (or
                    (not (arm-drone ?a))
                    (and
                        (in-formation ?a)
                        (at ?a ?dest)
                    )
                )
            )
        )
    :effect
        (and
            (mission-completed ?vtx)
            (forall (?a - drone)
                (when (arm-drone ?a)
                    (mission-completed ?a)))
            (parabola-mission-complete)
        )
)
;; Detect GPS failure
(:action detect-gps-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (gps-ok ?d))
        )
    :effect
        (and
            (gps-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect communication failure
(:action detect-communication-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (communication-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
            (not (swarm-comm-ready ?d))
        )
)
;; Detect loss of the drone-to-drone links - every link the drone has
;; drops with it, so the shape can't be re-formed until it is restored
(:action detect-neighbor-link-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (neighbor-comm-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
            (not (swarm-comm-ready ?d))
            (forall (?n - drone)
                (when (parabola-neighbor ?d ?n)
                    (and
                        (not (link-up ?d ?n))
                        (not (link-up ?n ?d))
                    )
                )
            )
        )
)
;; Detect when drone health becomes too low
(:action detect-health-failure
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
```

```
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/parabolaformation/problem.pddl

```
(define (problem parabola-formation-mission-001)
    (:domain parabola-formation-drone-mission)
    ;; Objects used in the parabola mission
    (:objects
        drone-vertex - drone
        drone-left   - drone
        drone-right  - drone
        ground-controller - controller
        source       - location
        destination  - location
    )
    (:init
        ;; All drones start landed at the source
        (at drone-vertex source)
        (at drone-left   source)
        (at drone-right  source)
        (landed drone-vertex)
        (landed drone-left)
        (landed drone-right)
        (= (altitude drone-vertex) 0)
        (= (altitude drone-left)   0)
        (= (altitude drone-right)  0)
        (source source)
        (destination destination)
        ;; Direct route between source and destination
        (connected source destination)
        (connected destination source)
        (safe-route source destination)
        (safe-route destination source)
        ;; The vertex leads from the middle; one arm drone either side
        (formation-leader drone-vertex)
        (slot-vertex drone-vertex)
        (arm-drone drone-left)
        (arm-drone drone-right)
        (has-slot drone-vertex)
        (has-slot drone-left)
        (has-slot drone-right)
        ;; All three on one line across the route, 10 m apart
        (= (slot-along-offset drone-vertex) 0)
        (= (slot-cross-offset drone-vertex) 0)
        (= (slot-along-offset drone-left)   0)
        (= (slot-cross-offset drone-left)  -10)
        (= (slot-along-offset drone-right)  0)
        (= (slot-cross-offset drone-right)  10)
        ;; Parabola shape - each arm 10 m above the vertex (50 m -> 60 m),
        ;; no two drones closer than 10 m
        (= (arm-rise) 10)
        (= (min-separation) 10)
        ;; Communication - every drone talks to both of the others: each
        ;; arm to the vertex, and the two arms to each other
        (parabola-neighbor drone-vertex drone-left)
        (parabola-neighbor drone-left drone-vertex)
        (parabola-neighbor drone-vertex drone-right)
        (parabola-neighbor drone-right drone-vertex)
        (parabola-neighbor drone-left drone-right)
        (parabola-neighbor drone-right drone-left)
        (neighbor-comm-ok drone-vertex)
        (neighbor-comm-ok drone-left)
        (neighbor-comm-ok drone-right)
        ;; Straight-line gaps in the formation: vertex to an arm is 10 m
        ;; across and 10 m up, arm to arm is 20 m across at the same height
        (= (link-distance drone-vertex drone-left)  14.142136)
        (= (link-distance drone-left drone-vertex)  14.142136)
        (= (link-distance drone-vertex drone-right) 14.142136)
        (= (link-distance drone-right drone-vertex) 14.142136)
        (= (link-distance drone-left drone-right)   20)
        (= (link-distance drone-right drone-left)   20)
        ;; Drone-to-drone link range
        (= (comm-range) 100)
        ;; All drones start with normal health and system status
        (gps-ok drone-vertex)
        (communication-ok drone-vertex)
```

```
        (drone-healthy drone-vertex)
        (gps-ok drone-left)
        (communication-ok drone-left)
        (drone-healthy drone-left)
        (gps-ok drone-right)
        (communication-ok drone-right)
        (drone-healthy drone-right)
        (= (health drone-vertex) 100)
        (= (health drone-left)   100)
        (= (health drone-right)  100)
        (= (minimum-health drone-vertex) 40)
        (= (minimum-health drone-left)   40)
        (= (minimum-health drone-right)  40)
        ;; Initial battery
        (= (battery drone-vertex) 100)
        (= (battery drone-left)   100)
        (= (battery drone-right)  100)
        (= (max-battery drone-vertex) 100)
        (= (max-battery drone-left)   100)
        (= (max-battery drone-right)  100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-vertex) 30)
        (= (minimum-return-battery drone-left)   30)
        (= (minimum-return-battery drone-right)  30)
        ;; Drones start at 2 m; the vertex climbs to 50 m, the arms to 60 m
        (= (hover-altitude drone-vertex) 2)
        (= (hover-altitude drone-left)   2)
        (= (hover-altitude drone-right)  2)
        (= (target-altitude drone-vertex) 50)
        (= (target-altitude drone-left)   60)
        (= (target-altitude drone-right)  60)
        ;; Battery used during the climb (the arms climb 10 m further)
        (= (climb-energy drone-vertex) 4)
        (= (climb-energy drone-left)   5)
        (= (climb-energy drone-right)  5)
        ;; Battery used by each drone for one cruise leg
        (= (cruise-leg-energy drone-vertex) 12)
        (= (cruise-leg-energy drone-left)   12)
        (= (cruise-leg-energy drone-right)  12)
        ;; Source and destination coordinates
        (= (latitude source)       18.3901547)
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
        ;; Direct route distance and energy cost
        (= (distance source destination) 3399.03)
        (= (distance destination source) 3399.03)
        (= (energy-required source destination) 10.00)
        (= (energy-required destination source) 10.00)
    )
    (:goal
        (and
            ;; Parabola is established
            (parabola-established)
            (in-formation drone-vertex)
            (in-formation drone-left)
            (in-formation drone-right)
            ;; Every drone has a working link with both of the others
            (swarm-comm-ready drone-vertex)
            (swarm-comm-ready drone-left)
            (swarm-comm-ready drone-right)
            ;; Vertex at 50 m, both arms at 60 m
            (at-cruise-altitude drone-vertex)
            (at-cruise-altitude drone-left)
            (at-cruise-altitude drone-right)
            (>= (altitude drone-vertex) 50)
            (>= (altitude drone-left)   60)
            (>= (altitude drone-right)  60)
            ;; Formation reached the destination
            (formation-at destination)
            (at drone-vertex destination)
            (at drone-left   destination)
            (at drone-right  destination)
            ;; Mission is completed
            (parabola-mission-complete)
            (mission-completed drone-vertex)
            (mission-completed drone-left)
            (mission-completed drone-right)
        )
    )
)
```

A.7 Figure-eight formation

plans/figure8formation/domain.pddl

```
(define (domain figure8-formation-drone-mission)
    ;; Figure-8 formation: drones fly from their base to the middle of a
    ;; figure-8 (two lobes crossing at a middle point), spread evenly round
    ;; the whole 8 at equal spacing, fly it a number of times, and come
    ;; back from the middle to the base
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
```

```
    :universal-preconditions
    :conditional-effects
)
;; Types used in the mission
(:types
    drone
    location
    controller
)
;; Facts that describe drone positions, states, and formation
(:predicates
    ;; Drone position
    (at ?d - drone ?l - location)
    ;; Current location of the whole train
    (formation-at ?l - location)
    ;; Mission locations: the base, and the figure-8 waypoint chain from
    ;; where a lap starts (the middle) to where it ends (the middle again)
    (base ?l - location)
    (lap-start ?l - location)
    (lap-end ?l - location)
    (connected ?from - location ?to - location)
    (safe-route ?from - location ?to - location)
    ;; The legs between the base and the middle of the 8
    (transit-route ?from - location ?to - location)
    ;; A waypoint where the two lobes of the 8 cross each other
    (crossing-point ?l - location)
    ;; Flight states
    (landed ?d - drone)
    (airborne ?d - drone)
    (flying ?d - drone)
    (at-cruise-altitude ?d - drone)
    ;; Train slots
    (train-leader ?d - drone)
    (train-follower ?d - drone)
    (has-slot ?d - drone)
    (in-formation ?d - drone)
    ;; Formation status
    (figure8-established)
    (train-closed)
    (pattern-entered)
    (laps-finished)
    ;; Neighbor communication - each drone talks to the drone directly
    ;; in front of it and directly behind it in the train
    (train-neighbor ?d1 - drone ?d2 - drone)
    (neighbor-comm-ok ?d - drone)
    ;; Pre-flight check
    (preflight-done ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
    (figure8-mission-complete)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Drone altitude
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    ;; Energy consumption
    (climb-energy ?d - drone)
    (cruise-leg-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    ;; Distance between locations (along the curve of the 8)
    (distance ?from - location ?to - location)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; Train slot: metres along the 8 behind the leader (negative = behind)
    (slot-arc-offset ?d - drone)
    ;; Spread geometry: the drones hold equal slots round the 8,
    ;; drone-spacing apart; crossing-gap is the smallest gap along the
    ;; path between two drones arriving at the middle crossing, which
    ;; must be at least crossing-clearance (10 m in a straight line)
    (drone-spacing)
    (crossing-gap)
    (crossing-clearance)
    ;; Laps of the 8: how many the mission asks for, how many are done
    (laps-required)
    (laps-completed)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
```

```
            (at ?d ?l)
            (base ?l)
            (landed ?d)
            (has-slot ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the drone's own slot on the 8 - every drone already
;; starts the spacing apart, so no staggered launch is needed
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (base ?l)
            (landed ?d)
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb from hover height to cruise altitude
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Move the drone into its assigned train slot
(:action take-train-slot
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (has-slot ?d)
            (at-cruise-altitude ?d)
            (>= (altitude ?d) (target-altitude ?d))
            (not (in-formation ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (in-formation ?d)
)
;; Establish the figure-8 formation when all drones are ready. The
;; equal spacing must never bring two drones to the middle crossing
;; point together: their gap along the path there must be at least the
;; crossing clearance (10 m apart in a straight line)
(:action establish-figure8-formation
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (train-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?l)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (neighbor-comm-ok ?lead)
            (>= (crossing-gap) (crossing-clearance))
            (forall (?f - drone)
                (or
                    (not (train-follower ?f))
                    (and
                        (in-formation ?f)
                        (at ?f ?l)
                        (at-cruise-altitude ?f)
                        (>= (altitude ?f) (target-altitude ?f))
                        (neighbor-comm-ok ?f)
```

```
                    )
                )
            )
        )
    :effect
        (and
            (figure8-established)
            (formation-at ?l)
            (train-closed)
        )
)
;; Fly the train from the base to the middle of the 8
(:action fly-to-pattern
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (figure8-established)
            (train-closed)
            (not (pattern-entered))
            (train-leader ?lead)
            (in-formation ?lead)
            (base ?from)
            (lap-start ?to)
            (transit-route ?from ?to)
            (formation-at ?from)
            (at ?lead ?from)
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
        )
    :effect
        (and
            (pattern-entered)
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            ;; Followers must catch up before the next leg
            (not (train-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Move the leader to the next waypoint of the 8
(:action formation-cruise
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (figure8-established)
            (train-closed)
            (pattern-entered)
            (not (laps-finished))
            (train-leader ?lead)
            (in-formation ?lead)
            (formation-at ?from)
            (at ?lead ?from)
            (connected ?from ?to)
            (safe-route ?from ?to)
            (at-cruise-altitude ?lead)
            (>= (altitude ?lead) (target-altitude ?lead))
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            ;; Followers must catch up before the next leg
            (not (train-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Move every follower along behind the leader
(:action follow-leader
    :parameters
        (?lead - drone
         ?to - location)
    :precondition
        (and
            (figure8-established)
            (not (train-closed))
            (train-leader ?lead)
            (in-formation ?lead)
            (formation-at ?to)
            (at ?lead ?to)
            (forall (?f - drone)
                (or
                    (not (train-follower ?f))
                    (and
                        (in-formation ?f)
```

```
                        (at-cruise-altitude ?f)
                        (>= (altitude ?f) (target-altitude ?f))
                        (gps-ok ?f)
                        (communication-ok ?f)
                        (drone-healthy ?f)
                        (neighbor-comm-ok ?f)
                        (> (battery ?f) (cruise-leg-energy ?f))
                    )
                )
            )
        )
    :effect
        (and
            (forall (?f - drone) (forall (?x - location)
                (when (and (train-follower ?f) (at ?f ?x))
                    (not (at ?f ?x)))))
            (forall (?f - drone)
                (when (train-follower ?f) (at ?f ?to)))
            (forall (?f - drone)
                (when (train-follower ?f)
                    (decrease (battery ?f) (cruise-leg-energy ?f))))
            (train-closed)
        )
)
;; Finish a lap at the crossing and start the next one from the same
;; point - only while more laps remain after the one being finished
(:action start-next-lap
    :parameters
        (?lead - drone
         ?dest - location
         ?start - location)
    :precondition
        (and
            (lap-end ?dest)
            (lap-start ?start)
            (figure8-established)
            (train-closed)
            (formation-at ?dest)
            (train-leader ?lead)
            (at ?lead ?dest)
            (< (+ (laps-completed) 1) (laps-required))
        )
    :effect
        (and
            (not (formation-at ?dest))
            (formation-at ?start)
            (not (at ?lead ?dest))
            (at ?lead ?start)
            (forall (?f - drone)
                (when (and (train-follower ?f) (at ?f ?dest))
                    (and (not (at ?f ?dest)) (at ?f ?start))))
            (increase (laps-completed) 1)
        )
)
;; Count the last lap as flown - the train is at the middle of the 8 and
;; no more laps remain
(:action finish-laps
    :parameters
        (?lead - drone
         ?end - location)
    :precondition
        (and
            (lap-end ?end)
            (figure8-established)
            (train-closed)
            (not (laps-finished))
            (formation-at ?end)
            (train-leader ?lead)
            (at ?lead ?end)
            (>= (+ (laps-completed) 1) (laps-required))
        )
    :effect
        (and
            (laps-finished)
            (increase (laps-completed) 1)
        )
)
;; Fly the train from the middle of the 8 back to the base
(:action return-to-base
    :parameters
        (?lead - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (laps-finished)
            (train-closed)
            (train-leader ?lead)
            (in-formation ?lead)
            (lap-end ?from)
            (base ?to)
            (transit-route ?from ?to)
            (formation-at ?from)
            (at ?lead ?from)
            (gps-ok ?lead)
            (communication-ok ?lead)
            (drone-healthy ?lead)
            (neighbor-comm-ok ?lead)
            (> (battery ?lead) (energy-required ?from ?to))
        )
    :effect
        (and
```

```
            (not (formation-at ?from))
            (formation-at ?to)
            (not (at ?lead ?from))
            (at ?lead ?to)
            ;; Followers must catch up before the mission can complete
            (not (train-closed))
            (decrease (battery ?lead) (energy-required ?from ?to))
        )
)
;; Complete the mission when the train has flown every lap and is back
;; at the base
(:action complete-figure8-mission
    :parameters
        (?lead - drone
         ?home - location)
    :precondition
        (and
            (base ?home)
            (laps-finished)
            (train-closed)
            (formation-at ?home)
            (train-leader ?lead)
            (in-formation ?lead)
            (at ?lead ?home)
            (forall (?f - drone)
                (or
                    (not (train-follower ?f))
                    (and
                        (in-formation ?f)
                        (at ?f ?home)
                    )
                )
            )
        )
    :effect
        (and
            (mission-completed ?lead)
            (forall (?f - drone)
                (when (train-follower ?f)
                    (mission-completed ?f)))
            (figure8-mission-complete)
        )
)
;; Detect GPS failure
(:action detect-gps-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (gps-ok ?d))
        )
    :effect
        (and
            (gps-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect communication failure
(:action detect-communication-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (communication-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect loss of the link to the front/back neighbor
(:action detect-neighbor-link-loss
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (not (neighbor-comm-ok ?d))
        )
    :effect
        (and
            (communication-lost ?d)
            (not (flying ?d))
            (not (in-formation ?d))
        )
)
;; Detect when drone health becomes too low
(:action detect-health-failure
    :parameters
        (?d - drone)
    :precondition
        (and
            (flying ?d)
            (< (health ?d) (minimum-health ?d))
        )
    :effect
        (and
```

```
                (health-failure ?d)
                (not (flying ?d))
                (not (in-formation ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/figure8formation/problem.pddl

```
(define (problem figure8-formation-mission-001)
    (:domain figure8-formation-drone-mission)
    ;; Objects used in the figure-8 mission
    (:objects
        drone-lead  - drone
        drone-f1    - drone
        ground-controller - controller
        ;; The base the drones fly from and back to
        base         - location
        ;; Key waypoints of the 8, in flying order: out of the middle into
        ;; the first lobe, round it, back through the middle, round the
        ;; second lobe and back to the middle (crossing-in and crossing-out
        ;; are both the middle point - the start and end of one lap)
        crossing-in  - location
        lobe1-a      - location
        lobe1-end    - location
        lobe1-b      - location
        crossing-mid - location
        lobe2-a      - location
        lobe2-end    - location
        lobe2-b      - location
        crossing-out - location
    )
    (:init
        ;; All drones start landed at the base
        (at drone-lead base)
        (at drone-f1 base)
        (landed drone-lead)
        (landed drone-f1)
        (= (altitude drone-lead) 0)
        (= (altitude drone-f1) 0)
        (base base)
        (lap-start crossing-in)
        (lap-end crossing-out)
        (crossing-point crossing-in)
        (crossing-point crossing-mid)
        (crossing-point crossing-out)
        ;; Legs between the base and the middle of the 8
        (transit-route base crossing-in)
        (transit-route crossing-out base)
        ;; One-way chain of waypoints round the 8
        (connected crossing-in lobe1-a)
        (connected lobe1-a lobe1-end)
        (connected lobe1-end lobe1-b)
        (connected lobe1-b crossing-mid)
        (connected crossing-mid lobe2-a)
        (connected lobe2-a lobe2-end)
        (connected lobe2-end lobe2-b)
        (connected lobe2-b crossing-out)
        (safe-route crossing-in lobe1-a)
        (safe-route lobe1-a lobe1-end)
        (safe-route lobe1-end lobe1-b)
        (safe-route lobe1-b crossing-mid)
        (safe-route crossing-mid lobe2-a)
        (safe-route lobe2-a lobe2-end)
        (safe-route lobe2-end lobe2-b)
        (safe-route lobe2-b crossing-out)
        ;; Train slots: the leader in front, followers one behind another
        (train-leader drone-lead)
        (train-follower drone-f1)
        (has-slot drone-lead)
        (has-slot drone-f1)
        ;; Metres along the path behind the leader
        (= (slot-arc-offset drone-lead) 0)
        (= (slot-arc-offset drone-f1) -10)
        ;; Neighbor communication - each drone talks to the drone directly
        ;; in front of it and directly behind it (here, just the one pair)
        (train-neighbor drone-lead drone-f1)
        (train-neighbor drone-f1 drone-lead)
        (neighbor-comm-ok drone-lead)
        (neighbor-comm-ok drone-f1)
        ;; Spread geometry - drones evenly spaced round the 8, never
        ;; meeting at the middle crossing closer than the clearance
        (= (drone-spacing) 304.86)
        (= (crossing-gap) 152.43)
        (= (crossing-clearance) 14.142136)
```

```
        ;; Laps of the 8 to fly (set from the operator's input at run time)
        (= (laps-required) 1)
        (= (laps-completed) 0)
        ;; All drones start with normal health and system status
        (gps-ok drone-lead)
        (communication-ok drone-lead)
        (drone-healthy drone-lead)
        (gps-ok drone-f1)
        (communication-ok drone-f1)
        (drone-healthy drone-f1)
        (= (health drone-lead) 100)
        (= (health drone-f1) 100)
        (= (minimum-health drone-lead) 40)
        (= (minimum-health drone-f1) 40)
        ;; Initial battery
        (= (battery drone-lead) 100)
        (= (battery drone-f1) 100)
        (= (max-battery drone-lead) 100)
        (= (max-battery drone-f1) 100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-lead) 30)
        (= (minimum-return-battery drone-f1) 30)
        ;; Drones start at 2 m and climb to 50 m
        (= (hover-altitude drone-lead) 2)
        (= (hover-altitude drone-f1) 2)
        (= (target-altitude drone-lead) 50)
        (= (target-altitude drone-f1) 50)
        ;; Battery used during the climb
        (= (climb-energy drone-lead) 4)
        (= (climb-energy drone-f1) 4)
        ;; Battery used by each follower for one leg
        (= (cruise-leg-energy drone-lead) 4)
        (= (cruise-leg-energy drone-f1) 4)
        ;; Coordinates (a 200 m-wide 8, the base 150 m below its middle)
        (= (latitude base)          18.3888072)
        (= (longitude base)         79.0495640)
        (= (latitude crossing-in)   18.3901547)
        (= (longitude crossing-in)  79.0495640)
        (= (latitude lobe1-a)       18.3906039)
        (= (longitude lobe1-a)      79.0502334)
        (= (latitude lobe1-end)     18.3901547)
        (= (longitude lobe1-end)    79.0505107)
        (= (latitude lobe1-b)       18.3897055)
        (= (longitude lobe1-b)      79.0502334)
        (= (latitude crossing-mid)  18.3901547)
        (= (longitude crossing-mid) 79.0495640)
        (= (latitude lobe2-a)       18.3906039)
        (= (longitude lobe2-a)      79.0488946)
        (= (latitude lobe2-end)     18.3901547)
        (= (longitude lobe2-end)    79.0486173)
        (= (latitude lobe2-b)       18.3897055)
        (= (longitude lobe2-b)      79.0488946)
        (= (latitude crossing-out)  18.3901547)
        (= (longitude crossing-out) 79.0495640)
        ;; Leg lengths along the path, and their energy cost
        (= (distance base crossing-in) 152.16)
        (= (distance crossing-in lobe1-a) 88.33)
        (= (distance lobe1-a lobe1-end) 64.10)
        (= (distance lobe1-end lobe1-b) 64.10)
        (= (distance lobe1-b crossing-mid) 88.33)
        (= (distance crossing-mid lobe2-a) 88.33)
        (= (distance lobe2-a lobe2-end) 64.10)
        (= (distance lobe2-end lobe2-b) 64.10)
        (= (distance lobe2-b crossing-out) 88.33)
        (= (distance crossing-out base) 198.21)
        (= (energy-required base crossing-in) 6.15)
        (= (energy-required crossing-in lobe1-a) 3.57)
        (= (energy-required lobe1-a lobe1-end) 2.59)
        (= (energy-required lobe1-end lobe1-b) 2.59)
        (= (energy-required lobe1-b crossing-mid) 3.57)
        (= (energy-required crossing-mid lobe2-a) 3.57)
        (= (energy-required lobe2-a lobe2-end) 2.59)
        (= (energy-required lobe2-end lobe2-b) 2.59)
        (= (energy-required lobe2-b crossing-out) 3.57)
        (= (energy-required crossing-out base) 8.01)
    )
    (:goal
        (and
            ;; Figure-8 train is established
            (figure8-established)
            (in-formation drone-lead)
            (in-formation drone-f1)
            ;; All drones reached 50 m cruise altitude
            (at-cruise-altitude drone-lead)
            (at-cruise-altitude drone-f1)
            (>= (altitude drone-lead) 50)
            (>= (altitude drone-f1) 50)
            ;; Every lap flown, and the train back at the base
            (laps-finished)
            (formation-at base)
            (at drone-lead base)
            (at drone-f1 base)
            ;; Mission is completed
            (figure8-mission-complete)
            (mission-completed drone-lead)
            (mission-completed drone-f1)
        )
    )
)
```

A.8 Star and circle drone show

plans/starformation/domain.pddl

```
(define (domain star-formation-drone-mission)
    ;; Star drone show round one place: the drones take off, form a circle,
    ;; then each lap form a star, a circle, spin the circle, form a star,
    ;; spin the star and form a circle again - and finally land where they
    ;; took off
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Facts that describe drone positions, states, and formation
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; The place the show is performed round
        (source ?l - location)
        ;; Flight states
        (landed ?d - drone)
        (airborne ?d - drone)
        (flying ?d - drone)
        (at-cruise-altitude ?d - drone)
        ;; Formation slots: the leader takes the star's top tip, every
        ;; other drone its own place round the star
        (formation-leader ?d - drone)
        (star-member ?d - drone)
        (has-slot ?d - drone)
        ;; The show's progress
        (show-started)
        (circle-formed)
        (star-formed)
        (lap-started)
        (circle-spun)
        (star-spun)
        ;; Neighbor communication - each drone and the two either side of it
        ;; round the circle
        (ring-neighbor ?d1 - drone ?d2 - drone)
        (neighbor-comm-ok ?d - drone)
        ;; Pre-flight check
        (preflight-done ?d - drone)
        ;; System health
        (gps-ok ?d - drone)
        (communication-ok ?d - drone)
        (drone-healthy ?d - drone)
        ;; Failure states
        (gps-lost ?d - drone)
        (communication-lost ?d - drone)
        (health-failure ?d - drone)
        (controller-notified ?d - drone)
        ;; Mission status
        (mission-completed ?d - drone)
        (star-mission-complete)
    )
    ;; Numeric values used by the planner
    (:functions
        ;; Battery
        (battery ?d - drone)
        (max-battery ?d - drone)
        (minimum-return-battery ?d - drone)
        ;; Drone altitude
        (altitude ?d - drone)
        (hover-altitude ?d - drone)
        (target-altitude ?d - drone)
        ;; Energy consumption
        (climb-energy ?d - drone)
        ;; Drone health
        (health ?d - drone)
        (minimum-health ?d - drone)
        ;; Location coordinates
        (latitude ?l - location)
        (longitude ?l - location)
        ;; Each drone's direction from the show's centre (degrees,
        ;; anticlockwise from the top tip) - descriptive
        (slot-angle ?d - drone)
        ;; Laps of the show: how many are asked for, how many are done
        (laps-required)
        (laps-completed)
    )
    ;; Check the drone before takeoff
    (:action pre-flight-check
        :parameters
            (?d - drone
             ?l - location)
        :precondition
            (and
```

```
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (not (show-started))
            (has-slot ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the drone's pad
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (not (show-started))
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb from hover height to the show's altitude
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Every drone spreads out from its pad onto the circle
(:action form-circle
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (at-cruise-altitude ?lead)
            (neighbor-comm-ok ?lead)
            (not (show-started))
            (forall (?m - drone)
                (or
                    (not (star-member ?m))
                    (and
                        (at-cruise-altitude ?m)
                        (neighbor-comm-ok ?m)
                    )
                )
            )
        )
    :effect
        (and
            (show-started)
            (circle-formed)
        )
)
;; Lap, step 1: the circle changes into the star
(:action form-star
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (circle-formed)
            (not (lap-started))
            (< (laps-completed) (laps-required))
            (gps-ok ?lead)
            (communication-ok ?lead)
        )
    :effect
        (and
            (not (circle-formed))
            (star-formed)
```

```
            (lap-started)
        )
)
;; Lap, step 2: the star changes back into the circle
(:action form-circle-in-lap
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (star-formed)
            (lap-started)
            (not (circle-spun))
        )
    :effect
        (and
            (not (star-formed))
            (circle-formed)
        )
)
;; Lap, step 3: the circle turns once round
(:action spin-circle
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (circle-formed)
            (lap-started)
            (not (circle-spun))
        )
    :effect
        (circle-spun)
)
;; Lap, step 4: the circle changes into the star again
(:action form-star-again
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (circle-formed)
            (circle-spun)
            (not (star-spun))
        )
    :effect
        (and
            (not (circle-formed))
            (star-formed)
        )
)
;; Lap, step 5: the whole star turns once round its centre
(:action spin-star
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (star-formed)
            (circle-spun)
            (not (star-spun))
        )
    :effect
        (star-spun)
)
;; Lap, step 6: the star changes back into the circle - the lap is done
(:action close-lap
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (star-formed)
            (star-spun)
        )
    :effect
        (and
            (not (star-formed))
            (circle-formed)
            (not (lap-started))
            (not (circle-spun))
            (not (star-spun))
            (increase (laps-completed) 1)
        )
)
;; Every lap flown: spread back in onto the pads and land
(:action land-show
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (source ?l)
            (formation-leader ?lead)
            (circle-formed)
            (not (lap-started))
            (>= (laps-completed) (laps-required))
        )
    :effect
        (and
            (not (circle-formed))
            (forall (?d - drone)
```

```
                    (when (has-slot ?d)
                        (and
                            (landed ?d)
                            (not (airborne ?d))
                            (not (flying ?d))
                            (at ?d ?l)
                            (mission-completed ?d))))
                (star-mission-complete)
            )
    )
    ;; Detect GPS failure
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect communication failure
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect loss of the link to a neighbor round the circle
    (:action detect-neighbor-link-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (neighbor-comm-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect when drone health becomes too low
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/starformation/problem.pddl

```
(define (problem star-formation-mission-001)
    (:domain star-formation-drone-mission)
    ;; Objects used in the star drone show
    (:objects
        drone-lead  - drone
        drone-m1    - drone
        ground-controller - controller
        source      - location
    )
    (:init
        ;; All drones start landed at the source
```

```
        (at drone-lead source)
        (at drone-m1 source)
        (landed drone-lead)
        (landed drone-m1)
        (= (altitude drone-lead) 0)
        (= (altitude drone-m1) 0)
        (source source)
        ;; Formation slots: the leader on the star's top tip
        (formation-leader drone-lead)
        (star-member drone-m1)
        (has-slot drone-lead)
        (has-slot drone-m1)
        (= (slot-angle drone-lead) 0)
        (= (slot-angle drone-m1) 60)
        ;; Neighbor communication - each drone and the two either side of it
        ;; round the circle (rebuilt for the real drone count at run time)
        (ring-neighbor drone-lead drone-m1)
        (ring-neighbor drone-m1 drone-lead)
        (neighbor-comm-ok drone-lead)
        (neighbor-comm-ok drone-m1)
        ;; Laps of the show (set from the operator's input at run time)
        (= (laps-required) 1)
        (= (laps-completed) 0)
        ;; All drones start with normal health and system status
        (gps-ok drone-lead)
        (communication-ok drone-lead)
        (drone-healthy drone-lead)
        (gps-ok drone-m1)
        (communication-ok drone-m1)
        (drone-healthy drone-m1)
        (= (health drone-lead) 100)
        (= (health drone-m1) 100)
        (= (minimum-health drone-lead) 40)
        (= (minimum-health drone-m1) 40)
        ;; Initial battery
        (= (battery drone-lead) 100)
        (= (battery drone-m1) 100)
        (= (max-battery drone-lead) 100)
        (= (max-battery drone-m1) 100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-lead) 30)
        (= (minimum-return-battery drone-m1) 30)
        ;; Drones start at 2 m and climb to 50 m
        (= (hover-altitude drone-lead) 2)
        (= (hover-altitude drone-m1) 2)
        (= (target-altitude drone-lead) 50)
        (= (target-altitude drone-m1) 50)
        ;; Battery used during the climb
        (= (climb-energy drone-lead) 4)
        (= (climb-energy drone-m1) 4)
        ;; Where the show is performed
        (= (latitude source)  18.3901547)
        (= (longitude source) 79.0495640)
    )
    (:goal
        (and
            ;; Every lap flown, every drone landed back at the source
            (star-mission-complete)
            (>= (laps-completed) (laps-required))
            (at drone-lead source)
            (at drone-m1 source)
            (mission-completed drone-lead)
            (mission-completed drone-m1)
            (landed drone-lead)
            (landed drone-m1)
        )
    )
)
```

A.9 Planetary orbit formation

plans/planets/domain.pddl

```
(define (domain planets-formation-drone-mission)
    ;; Solar-system drone show round one place (the sun): every drone flies
    ;; its own orbit, 50 m further out than the one inside it, neighbouring
    ;; orbits turning in opposite directions. A lap is one turn of the
    ;; OUTERMOST drone; inner drones go round more often in that time, and
    ;; every lap ends with all of them back on their starting line. After
    ;; the last lap every drone flies back in and lands where it took off
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Facts that describe drone positions, states, and orbits
    (:predicates
```

```
    ;; Drone position
    (at ?d - drone ?l - location)
    ;; The place the drones orbit round (the sun)
    (source ?l - location)
    ;; Flight states
    (landed ?d - drone)
    (airborne ?d - drone)
    (flying ?d - drone)
    (at-cruise-altitude ?d - drone)
    ;; Orbits: the leader flies the innermost one, every other drone
    ;; its own orbit further out
    (formation-leader ?d - drone)
    (planet ?d - drone)
    (has-orbit ?d - drone)
    ;; Which way round each drone orbits (seen from above)
    (orbits-clockwise ?d - drone)
    (orbits-anticlockwise ?d - drone)
    ;; Drones on neighbouring orbits (one 50 m step apart)
    (orbit-neighbor ?d1 - drone ?d2 - drone)
    ;; The show's progress
    (orbits-entered)
    ;; Neighbor communication - each drone and the drones on the orbits
    ;; either side of it
    (neighbor-comm-ok ?d - drone)
    ;; Pre-flight check
    (preflight-done ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
    (planets-mission-complete)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Drone altitude
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    ;; Energy consumption
    (climb-energy ?d - drone)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; Each drone's orbit radius round the source (metres) - descriptive
    (orbit-radius ?d - drone)
    ;; Laps (turns of the outermost orbit): asked for and done
    (laps-required)
    (laps-completed)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (not (orbits-entered))
            (has-orbit ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the drone's pad
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (not (orbits-entered))
            (preflight-done ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (and
```

```
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb from hover height to the show's altitude
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Every drone flies out from its pad onto its own orbit - only once
;; every drone is up, and only if no two neighbouring orbits turn the
;; same way
(:action enter-orbits
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (at-cruise-altitude ?lead)
            (neighbor-comm-ok ?lead)
            (not (orbits-entered))
            (forall (?m - drone)
                (or
                    (not (has-orbit ?m))
                    (and
                        (at-cruise-altitude ?m)
                        (neighbor-comm-ok ?m)
                    )
                )
            )
            (forall (?a - drone ?b - drone)
                (or
                    (not (orbit-neighbor ?a ?b))
                    (and (orbits-clockwise ?a) (orbits-anticlockwise ?b))
                    (and (orbits-anticlockwise ?a) (orbits-clockwise ?b))
                )
            )
        )
    :effect
        (orbits-entered)
)
;; One lap: the outermost drone goes once round its orbit, every inner
;; drone round its own as often as fits, and all are back on their
;; starting line together
(:action fly-lap
    :parameters
        (?lead - drone)
    :precondition
        (and
            (formation-leader ?lead)
            (orbits-entered)
            (< (laps-completed) (laps-required))
            (gps-ok ?lead)
            (communication-ok ?lead)
        )
    :effect
        (increase (laps-completed) 1)
)
;; Every lap flown: fly back in onto the pads and land
(:action land-planets
    :parameters
        (?lead - drone
         ?l - location)
    :precondition
        (and
            (source ?l)
            (formation-leader ?lead)
            (orbits-entered)
            (>= (laps-completed) (laps-required))
        )
    :effect
        (and
            (not (orbits-entered))
            (forall (?d - drone)
                (when (has-orbit ?d)
                    (and
                        (landed ?d)
                        (not (airborne ?d))
                        (not (flying ?d))
                        (at ?d ?l)
                        (mission-completed ?d))))
            (planets-mission-complete)
        )
)
```

```
    ;; Detect GPS failure
    (:action detect-gps-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect communication failure
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect loss of the link to a neighbouring orbit's drone
    (:action detect-neighbor-link-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (neighbor-comm-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect when drone health becomes too low
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/planets/problem.pddl

```
(define (problem planets-formation-mission-001)
    (:domain planets-formation-drone-mission)
    ;; Objects used in the solar-system show
    (:objects
        drone-mercury - drone
        drone-venus   - drone
        ground-controller - controller
        source        - location
    )
    (:init
        ;; All drones start landed at the source
        (at drone-mercury source)
        (at drone-venus source)
        (landed drone-mercury)
        (landed drone-venus)
        (= (altitude drone-mercury) 0)
        (= (altitude drone-venus) 0)
        (source source)
        ;; Orbits: the leader on the innermost, 50 m further out per drone
        ;; (radii and directions rebuilt for the real drone count at run time)
        (formation-leader drone-mercury)
```

```
        (planet drone-venus)
        (has-orbit drone-mercury)
        (has-orbit drone-venus)
        (= (orbit-radius drone-mercury) 50)
        (= (orbit-radius drone-venus) 100)
        (orbits-anticlockwise drone-mercury)
        (orbits-clockwise drone-venus)
        ;; Neighbouring orbits
        (orbit-neighbor drone-mercury drone-venus)
        (orbit-neighbor drone-venus drone-mercury)
        (neighbor-comm-ok drone-mercury)
        (neighbor-comm-ok drone-venus)
        ;; Laps of the outermost orbit (set from the operator's input at run time)
        (= (laps-required) 1)
        (= (laps-completed) 0)
        ;; All drones start with normal health and system status
        (gps-ok drone-mercury)
        (communication-ok drone-mercury)
        (drone-healthy drone-mercury)
        (gps-ok drone-venus)
        (communication-ok drone-venus)
        (drone-healthy drone-venus)
        (= (health drone-mercury) 100)
        (= (health drone-venus) 100)
        (= (minimum-health drone-mercury) 40)
        (= (minimum-health drone-venus) 40)
        ;; Initial battery
        (= (battery drone-mercury) 100)
        (= (battery drone-venus) 100)
        (= (max-battery drone-mercury) 100)
        (= (max-battery drone-venus) 100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone-mercury) 30)
        (= (minimum-return-battery drone-venus) 30)
        ;; Drones start at 2 m and climb to 50 m
        (= (hover-altitude drone-mercury) 2)
        (= (hover-altitude drone-venus) 2)
        (= (target-altitude drone-mercury) 50)
        (= (target-altitude drone-venus) 50)
        ;; Battery used during the climb
        (= (climb-energy drone-mercury) 4)
        (= (climb-energy drone-venus) 4)
        ;; Where the sun is
        (= (latitude source)  18.3901547)
        (= (longitude source) 79.0495640)
    )
    (:goal
        (and
            ;; Every lap flown, every drone landed back at the source
            (planets-mission-complete)
            (>= (laps-completed) (laps-required))
            (at drone-mercury source)
            (at drone-venus source)
            (mission-completed drone-mercury)
            (mission-completed drone-venus)
            (landed drone-mercury)
            (landed drone-venus)
        )
    )
)
```

A.10 Road inspection

plans/Inspect/domain.pddl

```
(define (domain road-inspection-drone-mission)
    ;; Road inspection: a drone follows a road from the source to the
    ;; destination, inspecting every segment of it on the way, then flies
    ;; back along the same road and lands at the source
    (:requirements
        :strips
        :typing
        :equality
        :numeric-fluents
        :negative-preconditions
        :disjunctive-preconditions
        :universal-preconditions
        :conditional-effects
    )
    ;; Types used in the mission
    (:types
        drone
        location
        controller
    )
    ;; Facts that describe the drone, the road and the inspection
    (:predicates
        ;; Drone position
        (at ?d - drone ?l - location)
        ;; Mission locations
        (source ?l - location)
        (destination ?l - location)
        ;; The road, as segments from the source end towards the destination
        (road-segment ?from - location ?to - location)
        (segment-inspected ?from - location ?to - location)
        (road-inspected)
        ;; The drone doing the inspection
        (inspector ?d - drone)
```

```
    ;; Flight states
    (landed ?d - drone)
    (airborne ?d - drone)
    (flying ?d - drone)
    (at-cruise-altitude ?d - drone)
    ;; Pre-flight check
    (preflight-done ?d - drone)
    ;; System health
    (gps-ok ?d - drone)
    (communication-ok ?d - drone)
    (drone-healthy ?d - drone)
    ;; Failure states
    (gps-lost ?d - drone)
    (communication-lost ?d - drone)
    (health-failure ?d - drone)
    (controller-notified ?d - drone)
    ;; Mission status
    (mission-completed ?d - drone)
)
;; Numeric values used by the planner
(:functions
    ;; Battery
    (battery ?d - drone)
    (max-battery ?d - drone)
    (minimum-return-battery ?d - drone)
    ;; Drone altitude
    (altitude ?d - drone)
    (hover-altitude ?d - drone)
    (target-altitude ?d - drone)
    ;; Energy consumption
    (climb-energy ?d - drone)
    (energy-required ?from - location ?to - location)
    ;; Length of a road segment
    (distance ?from - location ?to - location)
    ;; Drone health
    (health ?d - drone)
    (minimum-health ?d - drone)
    ;; Location coordinates
    (latitude ?l - location)
    (longitude ?l - location)
    ;; How much of the road is inspected
    (segments-total)
    (segments-inspected)
)
;; Check the drone before takeoff
(:action pre-flight-check
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (inspector ?d)
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (not (road-inspected))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (>= (health ?d) (minimum-health ?d))
            (> (battery ?d) (minimum-return-battery ?d))
        )
    :effect
        (preflight-done ?d)
)
;; Take off from the source end of the road
(:action takeoff
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (inspector ?d)
            (at ?d ?l)
            (source ?l)
            (landed ?d)
            (preflight-done ?d)
            (not (road-inspected))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
        )
    :effect
        (and
            (not (landed ?d))
            (airborne ?d)
            (flying ?d)
            (assign (altitude ?d) (hover-altitude ?d))
        )
)
;; Climb from hover height to inspection altitude
(:action climb-to-cruise-altitude
    :parameters
        (?d - drone)
    :precondition
        (and
            (airborne ?d)
            (not (at-cruise-altitude ?d))
            (< (altitude ?d) (target-altitude ?d))
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
```

```
            (> (battery ?d) (climb-energy ?d))
        )
    :effect
        (and
            (assign (altitude ?d) (target-altitude ?d))
            (at-cruise-altitude ?d)
            (decrease (battery ?d) (climb-energy ?d))
        )
)
;; Fly along the next segment of the road, inspecting it
(:action inspect-segment
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (inspector ?d)
            (road-segment ?from ?to)
            (not (segment-inspected ?from ?to))
            (not (road-inspected))
            (at ?d ?from)
            (at-cruise-altitude ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (drone-healthy ?d)
            (> (battery ?d) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (at ?d ?from))
            (at ?d ?to)
            (segment-inspected ?from ?to)
            (increase (segments-inspected) 1)
            (decrease (battery ?d) (energy-required ?from ?to))
        )
)
;; At the destination with every segment inspected: the road is done
(:action finish-inspection
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (inspector ?d)
            (destination ?l)
            (at ?d ?l)
            (>= (segments-inspected) (segments-total))
        )
    :effect
        (road-inspected)
)
;; Fly back along the same road, one segment at a time
(:action return-along-road
    :parameters
        (?d - drone
         ?from - location
         ?to - location)
    :precondition
        (and
            (inspector ?d)
            (road-inspected)
            (road-segment ?to ?from)
            (at ?d ?from)
            (at-cruise-altitude ?d)
            (gps-ok ?d)
            (communication-ok ?d)
            (> (battery ?d) (energy-required ?from ?to))
        )
    :effect
        (and
            (not (at ?d ?from))
            (at ?d ?to)
            (decrease (battery ?d) (energy-required ?from ?to))
        )
)
;; Land back at the source
(:action land-at-source
    :parameters
        (?d - drone
         ?l - location)
    :precondition
        (and
            (inspector ?d)
            (road-inspected)
            (source ?l)
            (at ?d ?l)
            (airborne ?d)
        )
    :effect
        (and
            (not (airborne ?d))
            (not (flying ?d))
            (not (at-cruise-altitude ?d))
            (landed ?d)
            (mission-completed ?d)
        )
)
;; Detect GPS failure
(:action detect-gps-loss
    :parameters
        (?d - drone)
```

```
        :precondition
            (and
                (flying ?d)
                (not (gps-ok ?d))
            )
        :effect
            (and
                (gps-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect communication failure
    (:action detect-communication-loss
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (not (communication-ok ?d))
            )
        :effect
            (and
                (communication-lost ?d)
                (not (flying ?d))
            )
    )
    ;; Detect when drone health becomes too low
    (:action detect-health-failure
        :parameters
            (?d - drone)
        :precondition
            (and
                (flying ?d)
                (< (health ?d) (minimum-health ?d))
            )
        :effect
            (and
                (health-failure ?d)
                (not (flying ?d))
            )
    )
    ;; Inform the ground controller about a failure
    (:action notify-ground-controller
        :parameters
            (?d - drone
             ?c - controller)
        :precondition
            (and
                (or
                    (gps-lost ?d)
                    (communication-lost ?d)
                    (health-failure ?d)
                )
            )
        :effect
            (controller-notified ?d)
    )
)
```

plans/Inspect/problem.pddl

```
(define (problem road-inspection-mission-001)
    (:domain road-inspection-drone-mission)
    ;; Objects used in the road inspection
    (:objects
        drone1      - drone
        ground-controller - controller
        ;; The road's two ends (the waypoints between them are added at run
        ;; time from the real road's route)
        source      - location
        destination - location
    )
    (:init
        ;; The drone starts landed at the source end of the road
        (at drone1 source)
        (landed drone1)
        (inspector drone1)
        (= (altitude drone1) 0)
        (source source)
        (destination destination)
        ;; The road, as one segment here - replaced at run time by the real
        ;; road's segments
        (road-segment source destination)
        (= (distance source destination) 1000.00)
        (= (distance destination source) 1000.00)
        (= (energy-required source destination) 20.00)
        (= (energy-required destination source) 20.00)
        (= (segments-total) 1)
        (= (segments-inspected) 0)
        ;; The drone starts with normal health and system status
        (gps-ok drone1)
        (communication-ok drone1)
        (drone-healthy drone1)
        (= (health drone1) 100)
        (= (minimum-health drone1) 40)
        ;; Initial battery
        (= (battery drone1) 100)
        (= (max-battery drone1) 100)
        ;; Battery level for emergency return
        (= (minimum-return-battery drone1) 30)
        ;; The drone starts at 2 m and climbs to 50 m to inspect
```

```
        (= (hover-altitude drone1) 2)
        (= (target-altitude drone1) 50)
        ;; Battery used during the climb
        (= (climb-energy drone1) 4)
        ;; The road's two ends
        (= (latitude source)       18.3901547)
        (= (longitude source)      79.0495640)
        (= (latitude destination)  18.3876026)
        (= (longitude destination) 79.0816646)
    )
    (:goal
        (and
            ;; The whole road inspected, and the drone back and landed at the source
            (road-inspected)
            (at drone1 source)
            (landed drone1)
            (mission-completed drone1)
        )
    )
)
```